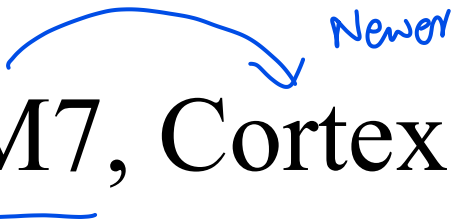


Embedded System CPUs: ARM7, Cortex M3



COE718: Embedded Systems Design

<http://www.ecb.torontomu.ca/~courses/coe718/>

Dr. Gul N. Khan

<http://www.ecb.torontomu.ca/~gnkhan>

Electrical, Computer and Biomedical Engineering

Toronto Metropolitan University

Overview

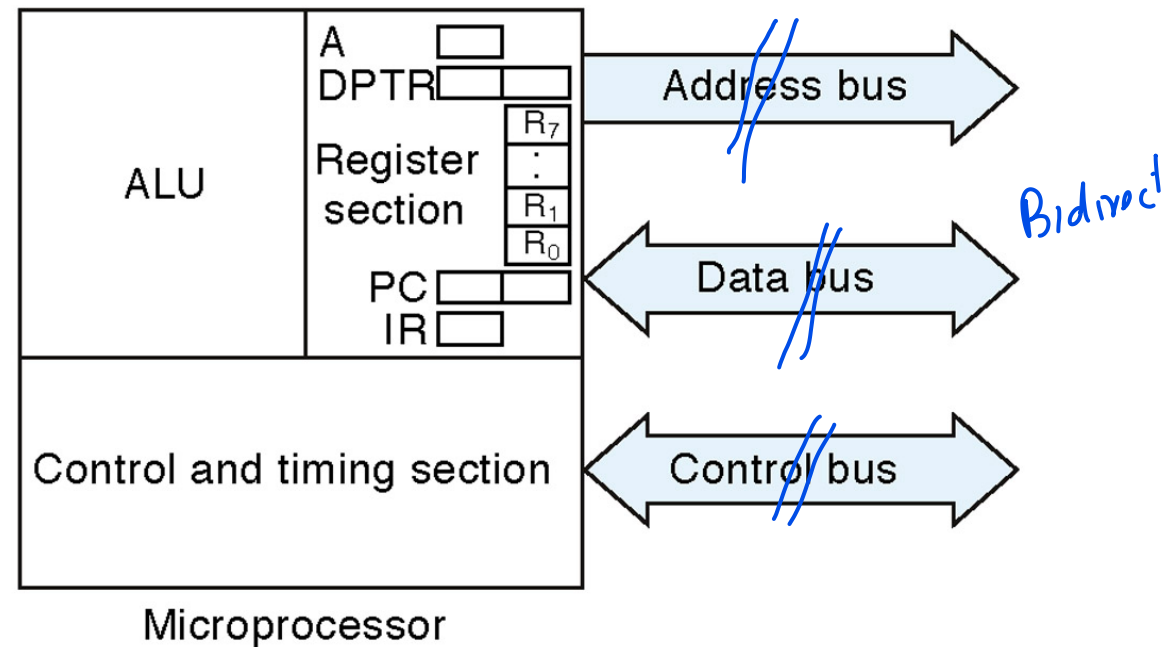
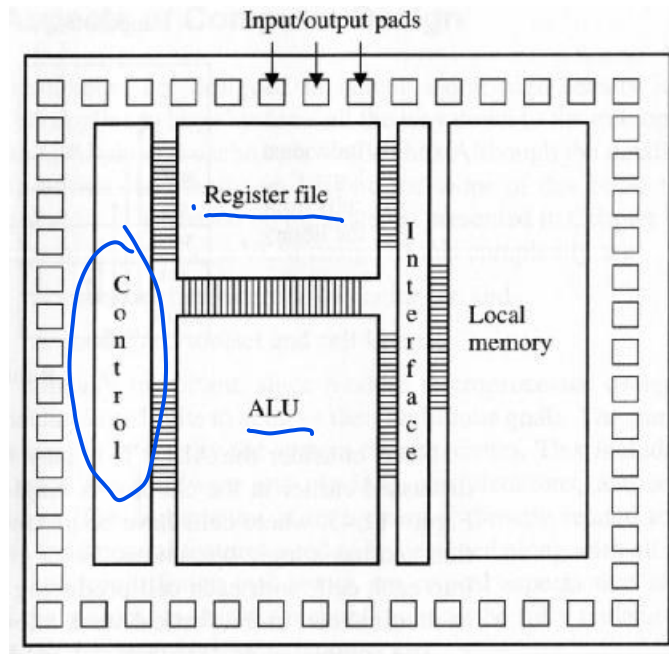
- Processors and System Architecture, Interrupts, Memory System
- Pipelining, I/O and CPU Performance
- Micro-controllers and Embedded CPUs
- ARM Architectures: ARM7TDMI
- Cortex-M3 a small foot-print Microcontroller

Text by Lewis: Chapter 5, part of Chapters 6 and 8 (8.1) and ARM7/M3 Data Sheets

Text by M. Wolf: part of Chapters/Sections 2.1, 2.2, 2.3 and 3.1-3.5

CPU/Processor Architecture

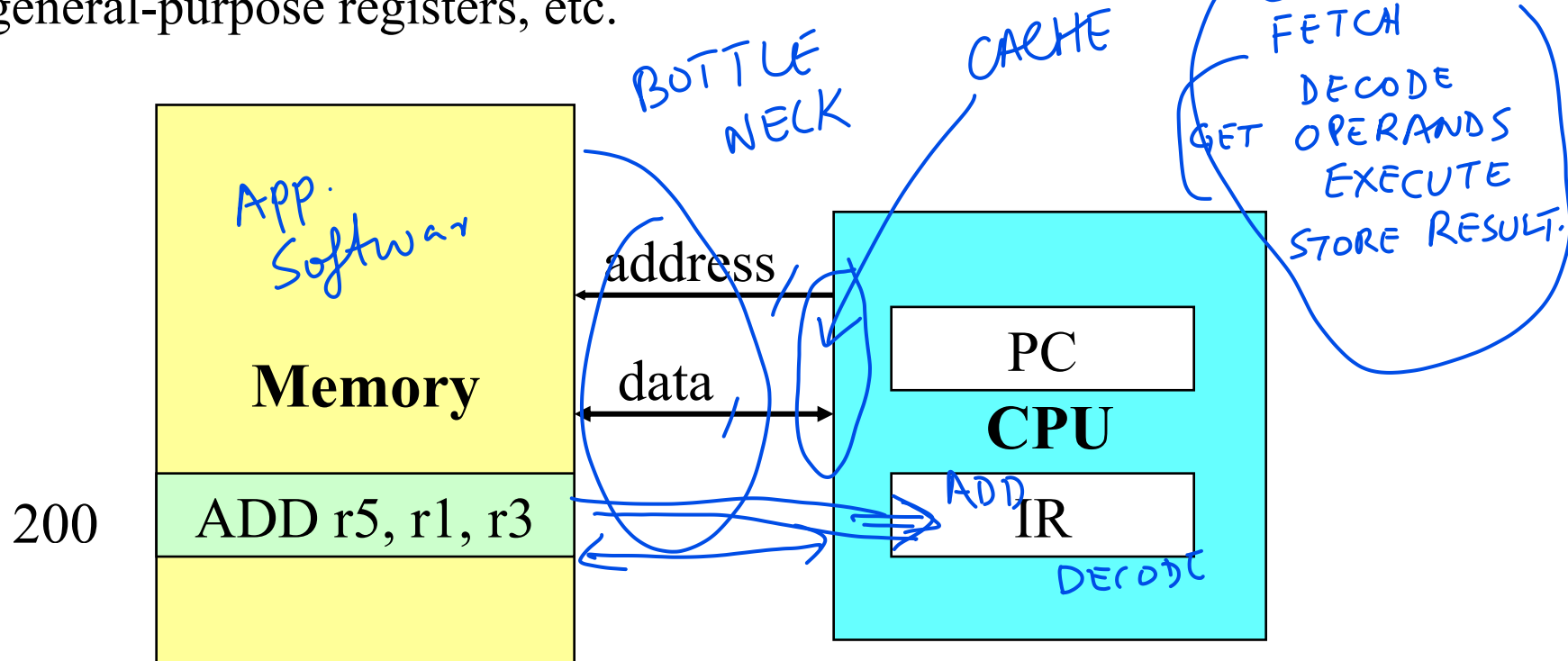
- Control and Timing Section
- Register Section
- ALU (Arithmetic Logic Unit)



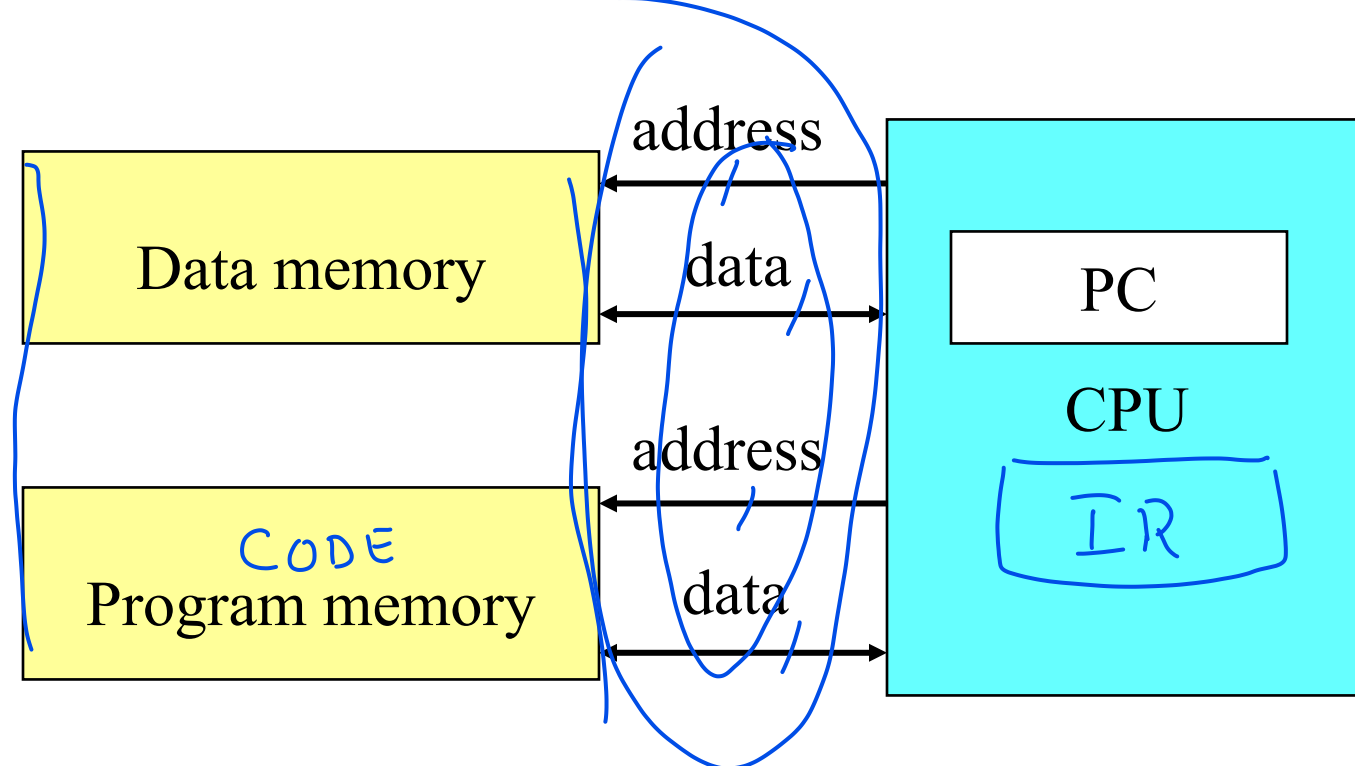
Processor Architectures

von Neumann architecture

- Memory holds data, instructions.
- Central processing unit (CPU) fetches instructions from memory.
- Separate CPU and memory distinguishes programmable computer.
- CPU registers help out: program counter (PC), instruction register (IR), general-purpose registers, etc.



Harvard Architecture



- Harvard architecture cannot use self-modifying code.
- It allows two simultaneous memory fetches.
- Most DSPs use Harvard architecture for streaming data.

Instruction Execution Process

Instruction Fetch: Reads next instruction into the instruction register (IR). PC has the instruction address.

Instruction Interpretation: Decodes the op-code, gets the required operands and routes them to ALU.

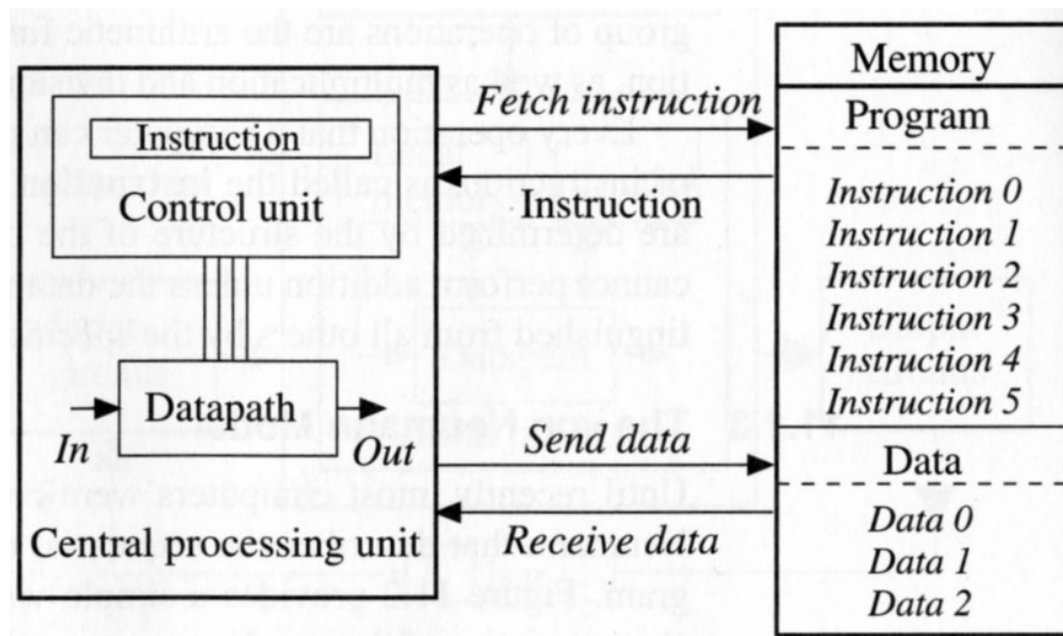
Sequencing

Determines the address of next instruction and loads it into the PC.

Execution: Generates control signals of ALU for execution.

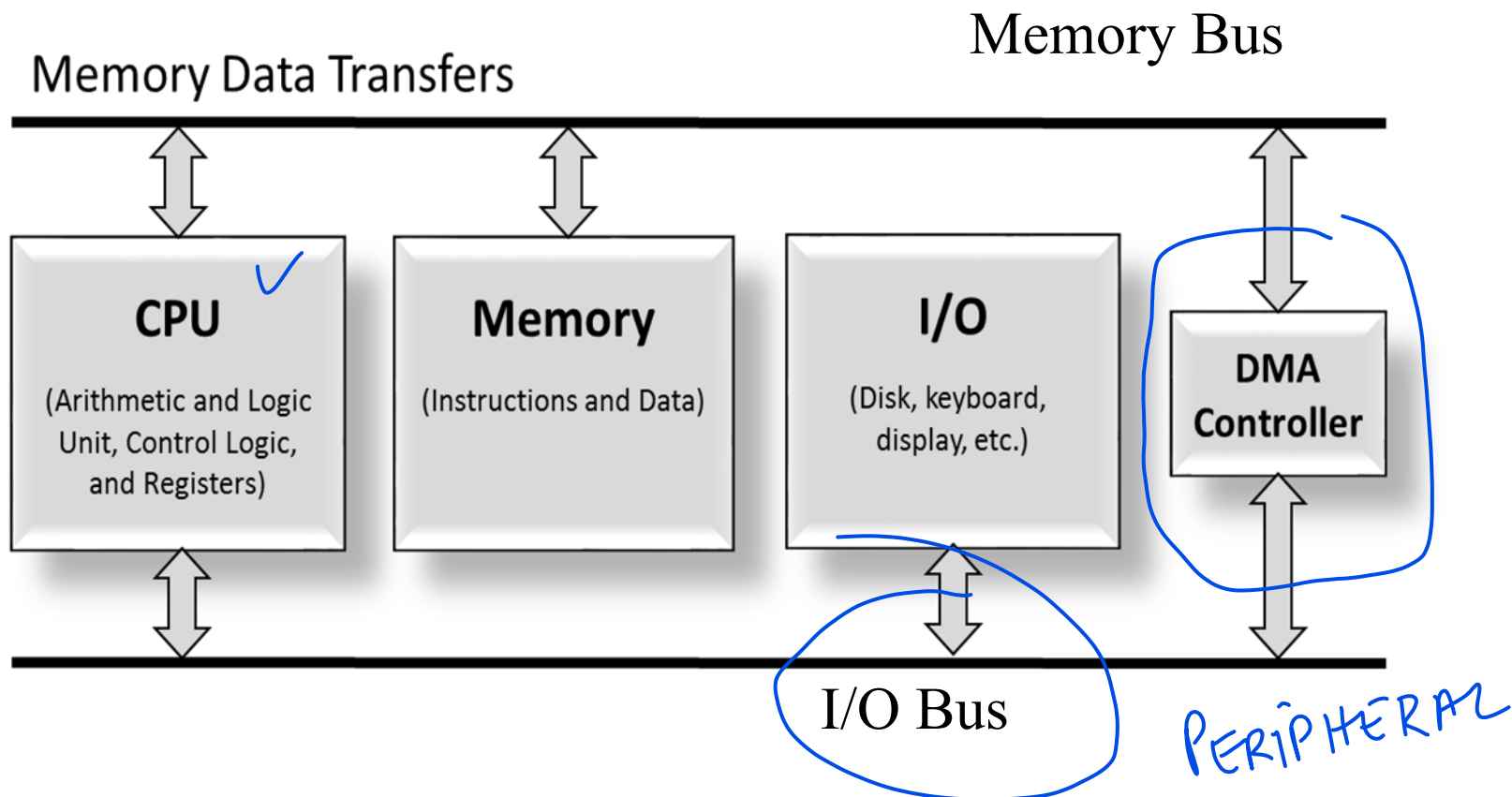
$$PC \leftarrow PC + 4$$

32-bit
CPU

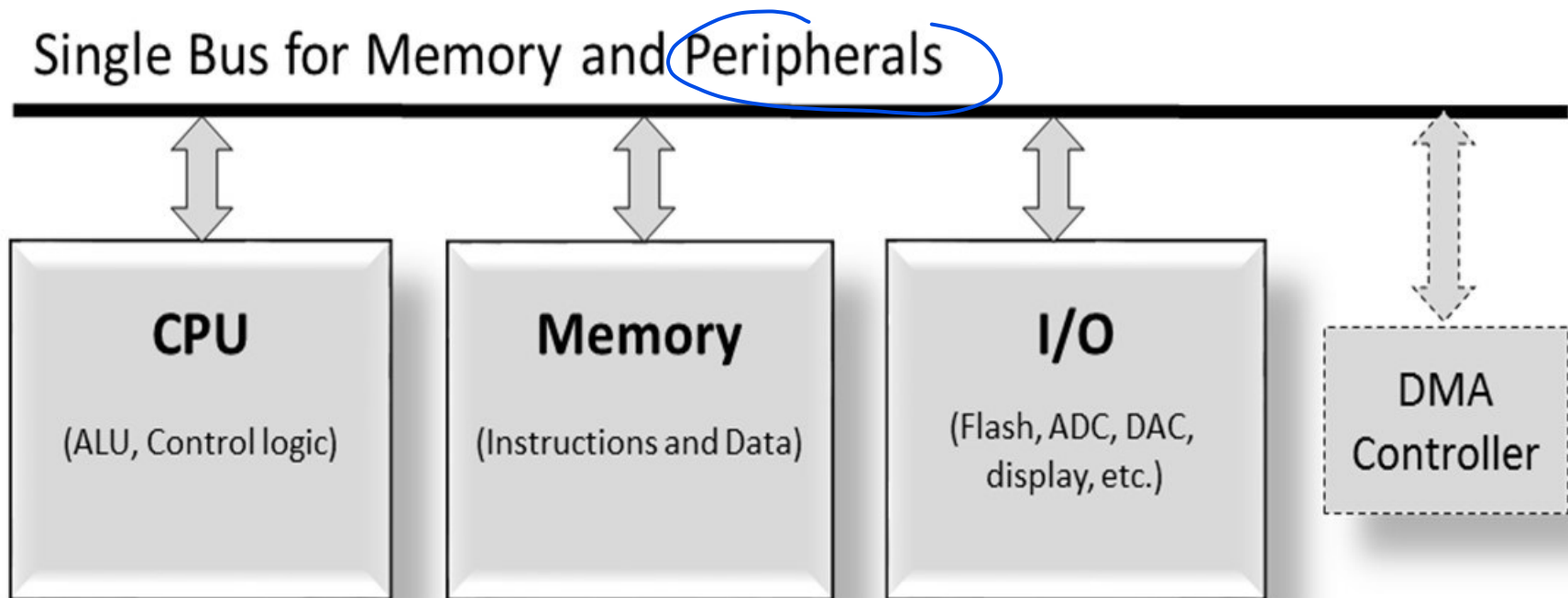


System Organization

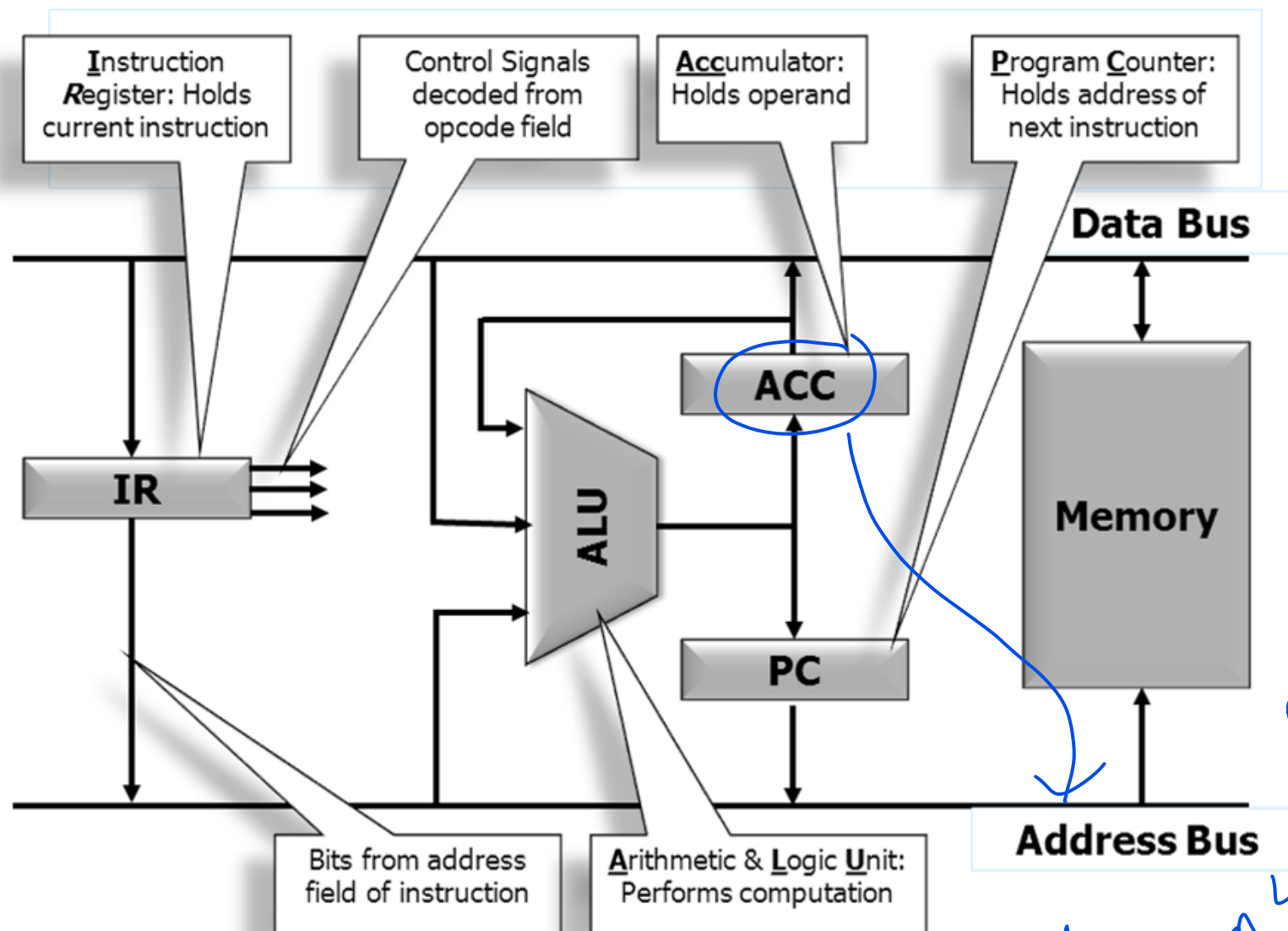
Memory and I/O having Separate Bus



Memory Mapped Peripherals



Single Accumulator Architecture



Interrupts

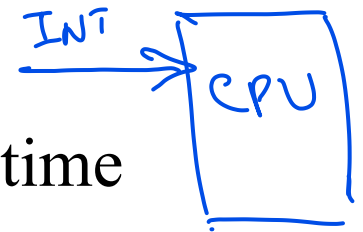
A computer program has only two ways to determine the conditions that exist in internal and external circuits.

to The Embedded SYSTEM

- One method uses software instructions that jump to subroutine on some flag status.
- The second method responds to hardware signals called interrupts that force the program to call interrupt-handling subroutines.
- Interrupts take processor time only when action is required.
- Processor can respond to an external event much faster by using interrupts.

ISR

The whole programming of microcomputers and micro-controller by using interrupts is called real-time programming.



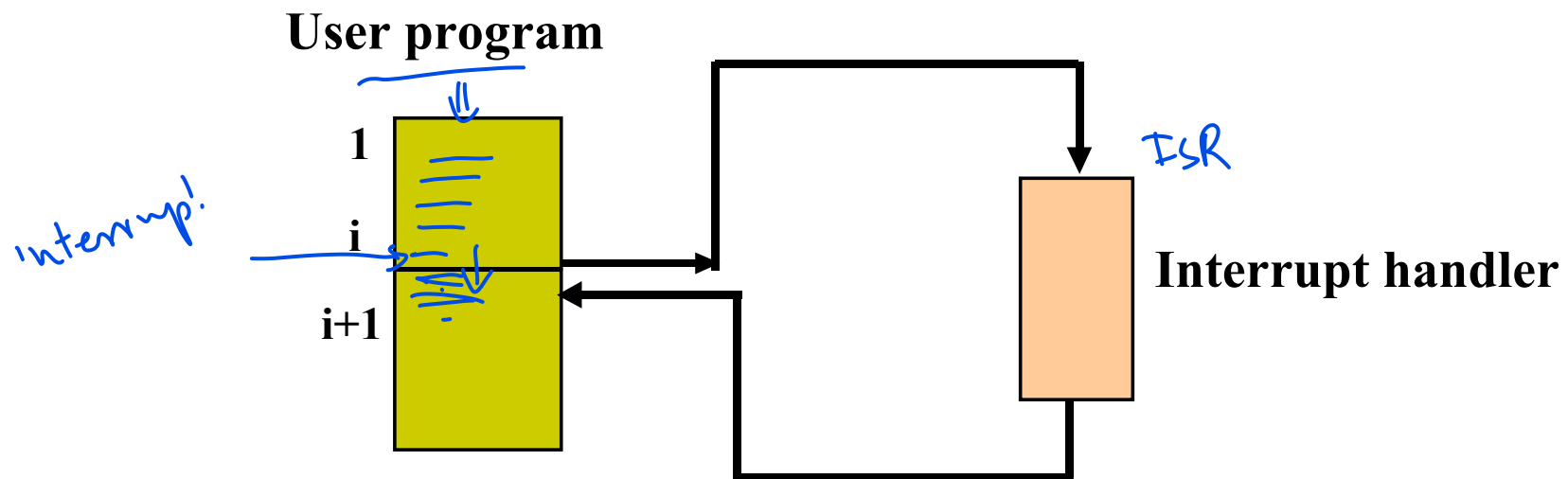
Interrupts are often the only way for successful real-time programming.

Instruction Cycle with Interrupts

Generally CPU checks for interrupts at the end of each instruction and executes the interrupt handler if required.

Interrupt Handler program identifies the nature/source of an interrupt and performs whatever actions are needed.

- It takes over the control after the interrupt.
- Control is transferred back to the interrupted program that will resume execution from the point of interruption.
- Point of interruption can occur anywhere in a program.
- State of the program is saved.



Interrupt Processing

Hardware

Device controller or other system h/w issues an interrupt

Processor finishes execution of current instruction

Processor signals acknowledgment of interrupt

Processor pushes PSW and PC onto control stack

Processor loads new PC value based on the interrupt

Software

Save remainder of process state information

Process interrupt

Restore process state information

Restore old PSW, PC, etc.

RESPONSE TIME

ISR

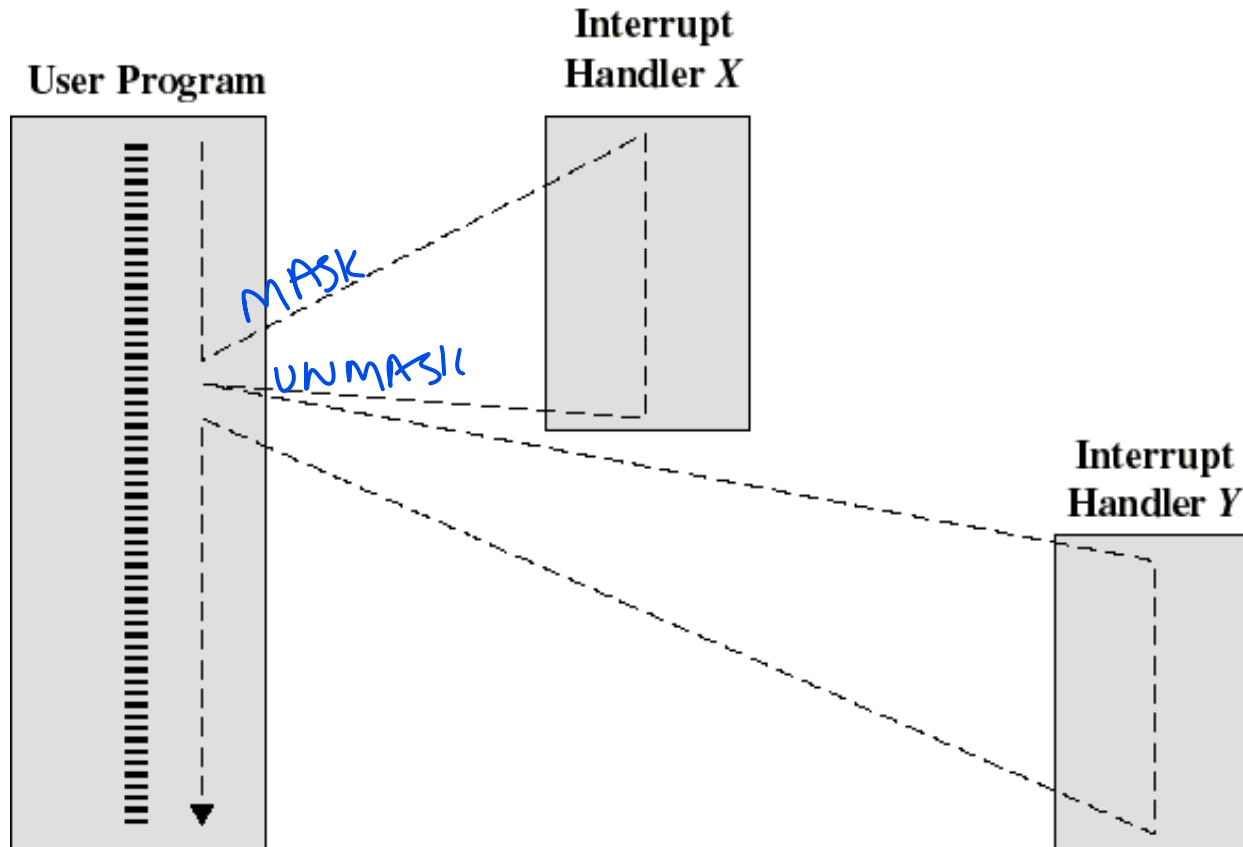
TASK

store

ISR

Multiple Interrupts (Sequential Order)

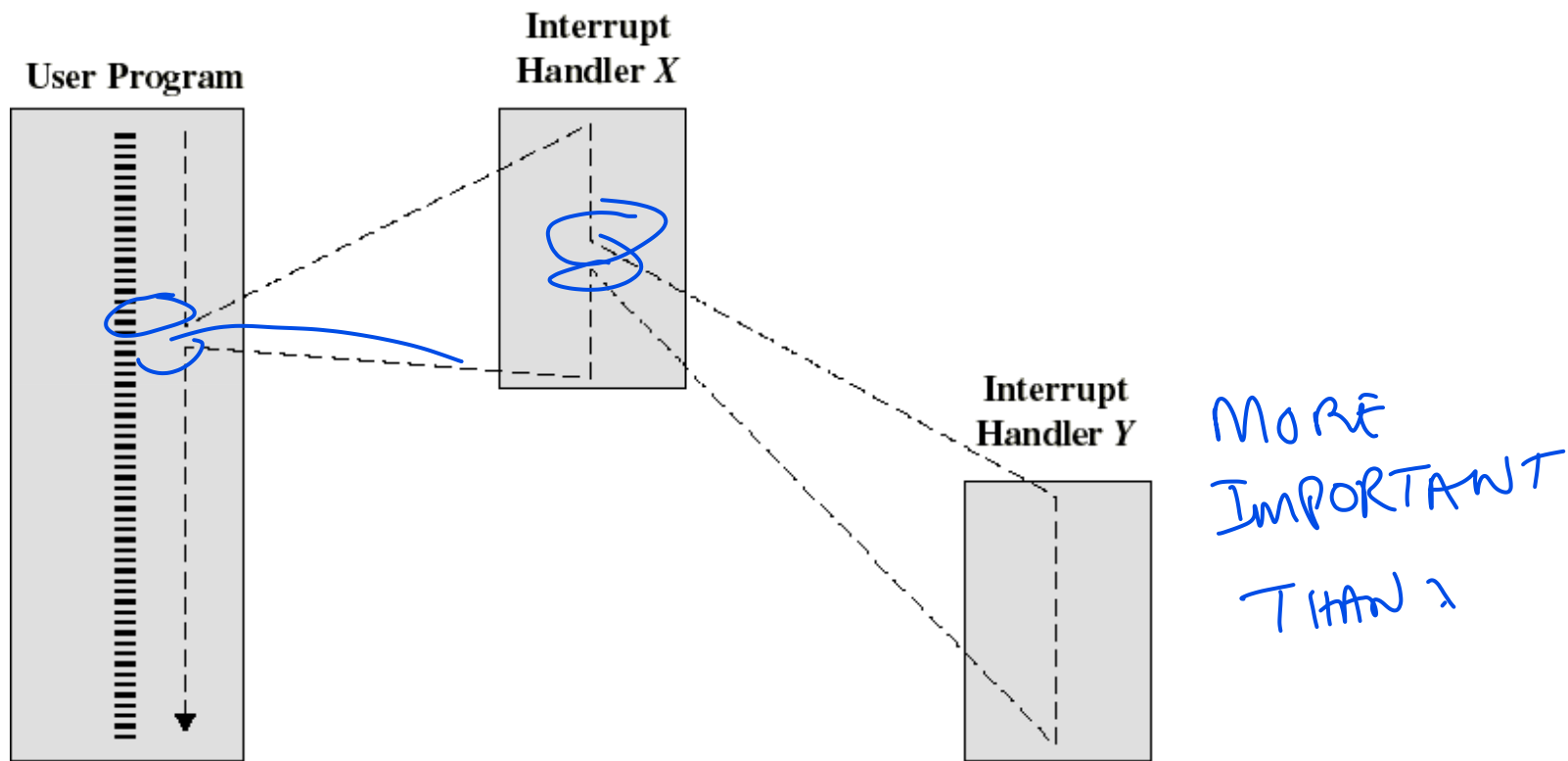
- Disable interrupts to complete the interrupting task at hand.
- Additional interrupts remain pending until interrupts are enabled. Then interrupts are considered in order
- After completing the interrupt handler routine, the processor checks for additional interrupts.



Multiple Interrupts (Nested)

- A higher priority interrupt causes lower-priority interrupts to wait.
- A lower-priority interrupt handler is interrupted.

For example, when input arrives from a communication line, it needs to be absorbed quickly to make room for additional inputs.



Processor Operation Modes

User mode

Application

- A user program is running.
- Certain instructions are not allowed.
- Memory mapping (base and bound) is enabled.

Supervisor mode

OS required / SPECIAL
SYSTEM

- The operating system is running.
- All instructions are allowed.
- Memory mapping (base and bound) is disabled.

OS
RTOS

A single PSW (processor status word) bit sets the above two modes:

For instance: PSW-bit = 1 for Supervisor mode

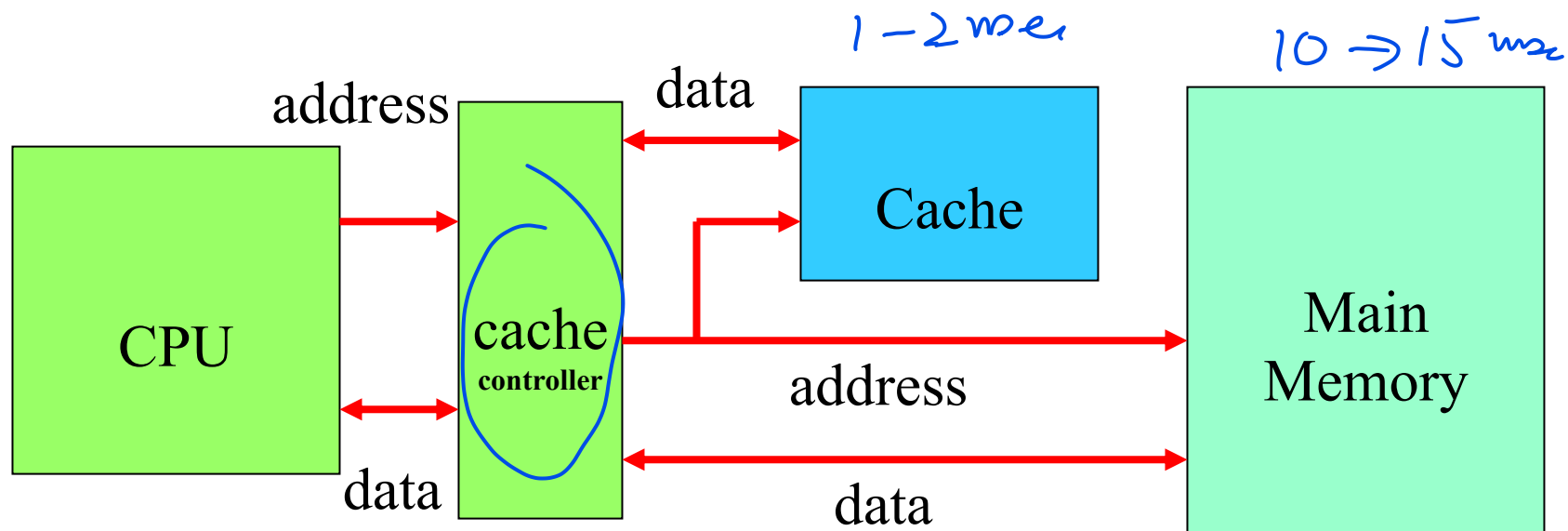
OS
controlled

Cache Memory

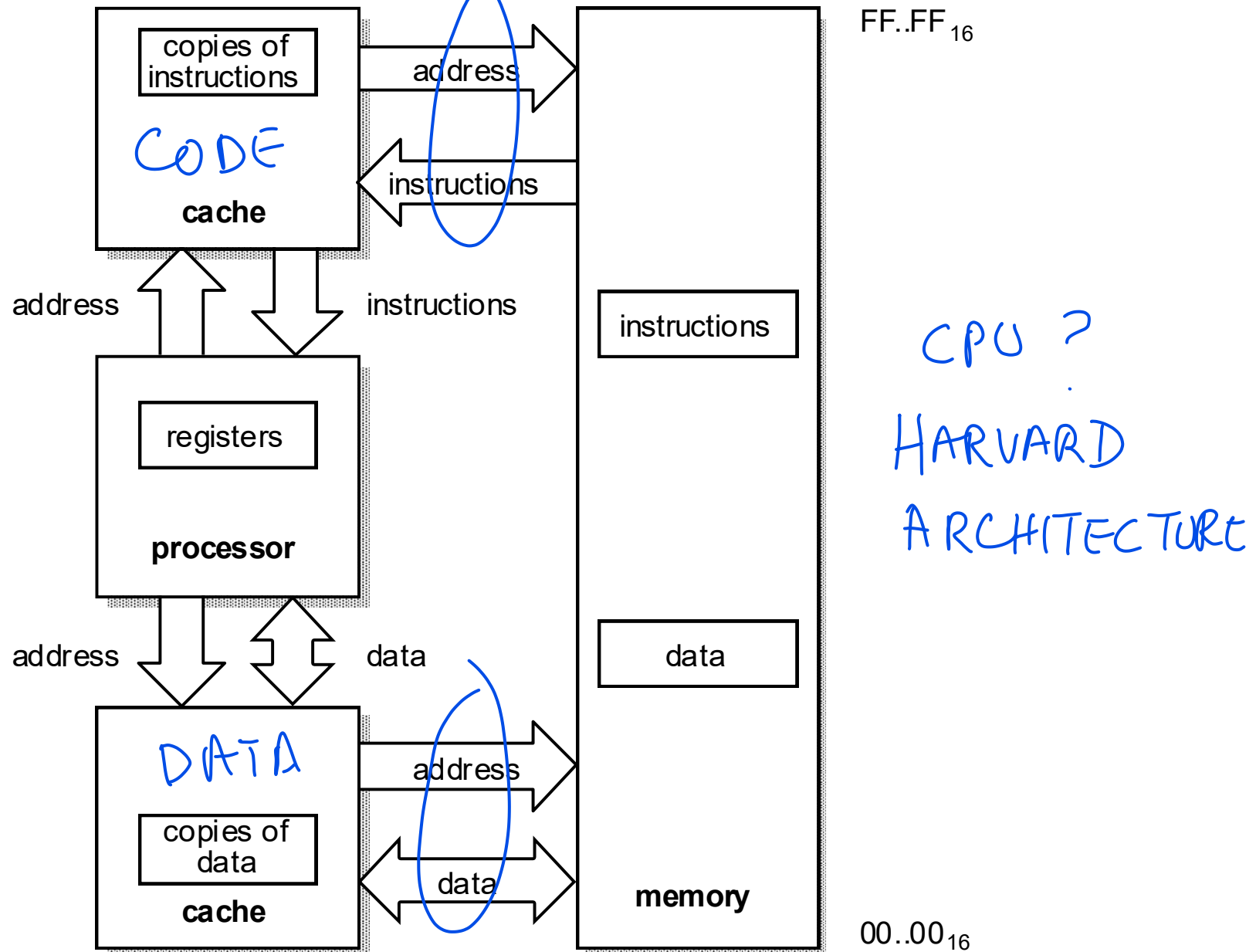
COE758

1-2 nsec
Access

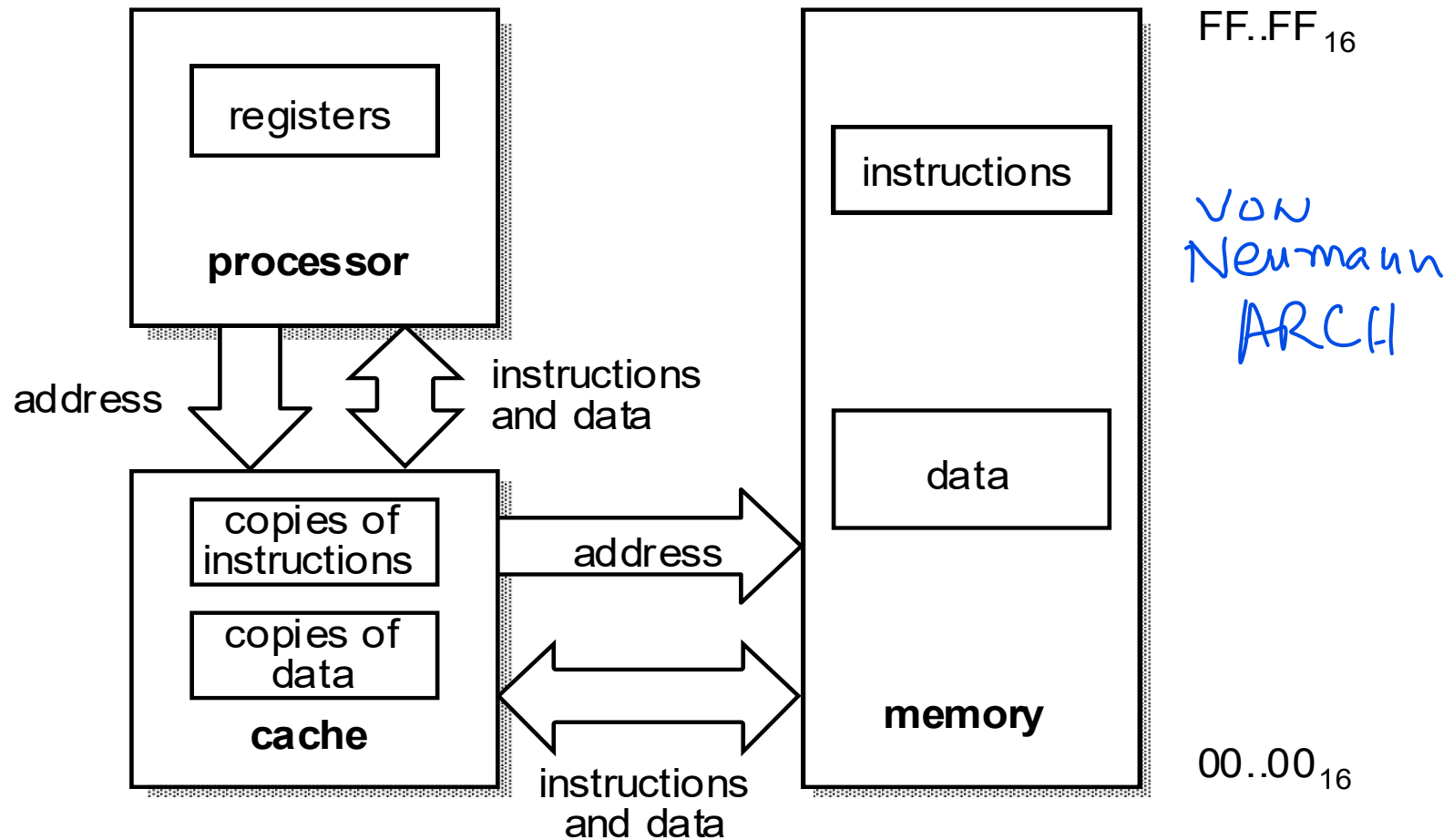
- Cache: Expensive but very fast memory directly connected to CPU interacting with slower but much larger main memory.
- Processor first checks if the addresses word is in cache.
(DATA / CODE)
- If the word is not found in cache, a block of memory containing the word is moved to the cache.



Separate Data and Instruction Caches

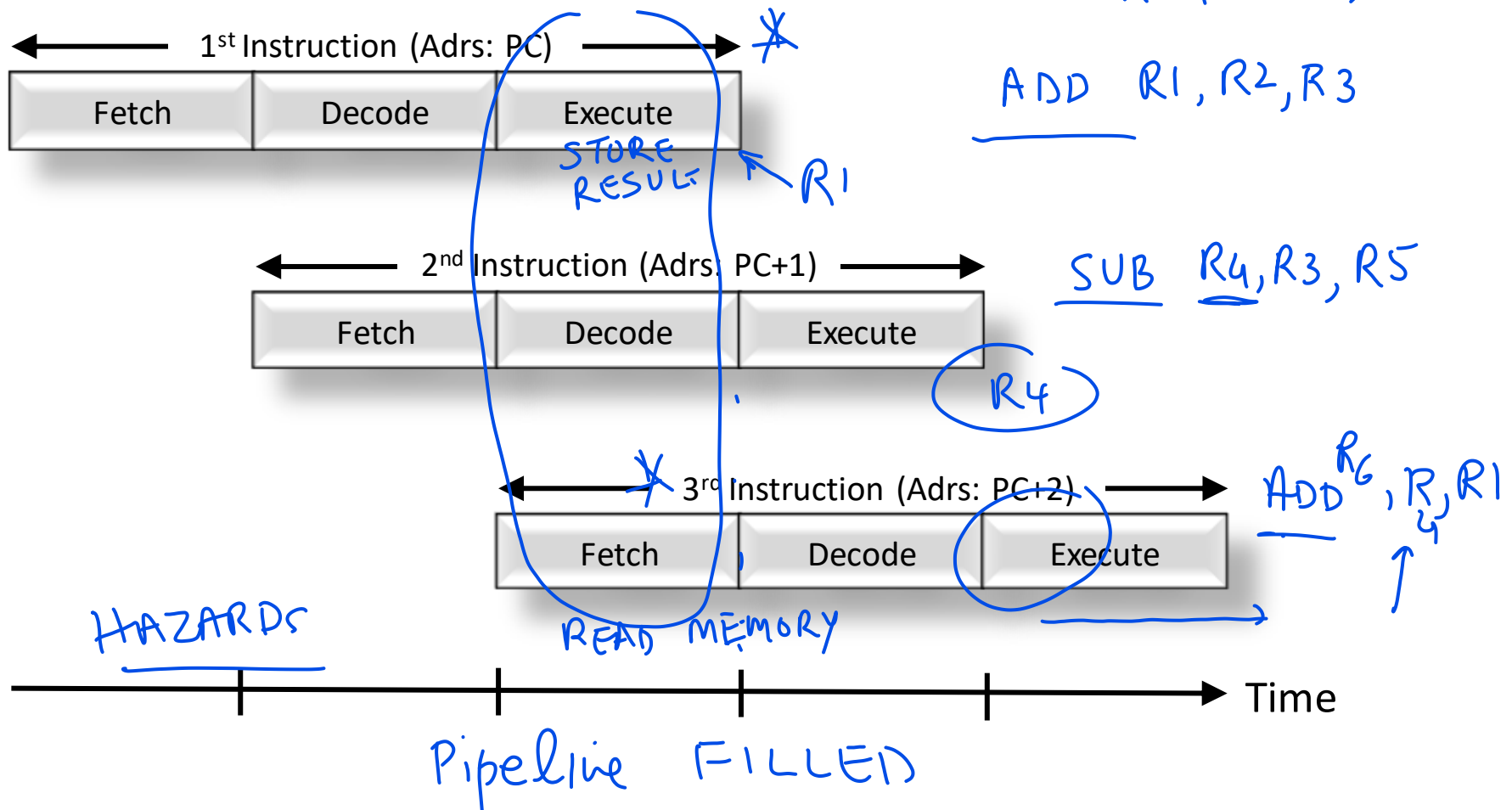


A Unified Instruction and Data Cache



CPU Pipelining

Improve performance by increasing instruction throughput.



CPU Pipelining

What makes pipelining easy?

- When all instructions are of the same length.
- Few instruction formats.
- Memory operands appear only in loads and stores.

RISC

R3000

What makes pipelining hard?

- Structural Hazards:

Slows down

68HC12
[CISC]

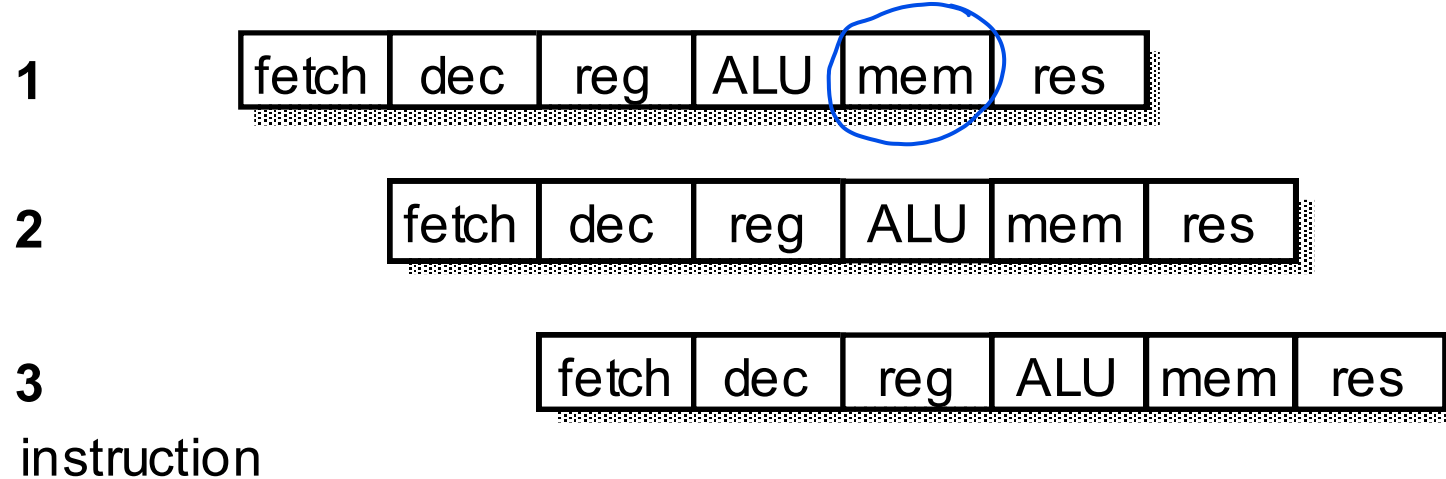
Fewer in Harvard Arch^{7, 15, 13}

- Control Hazards:

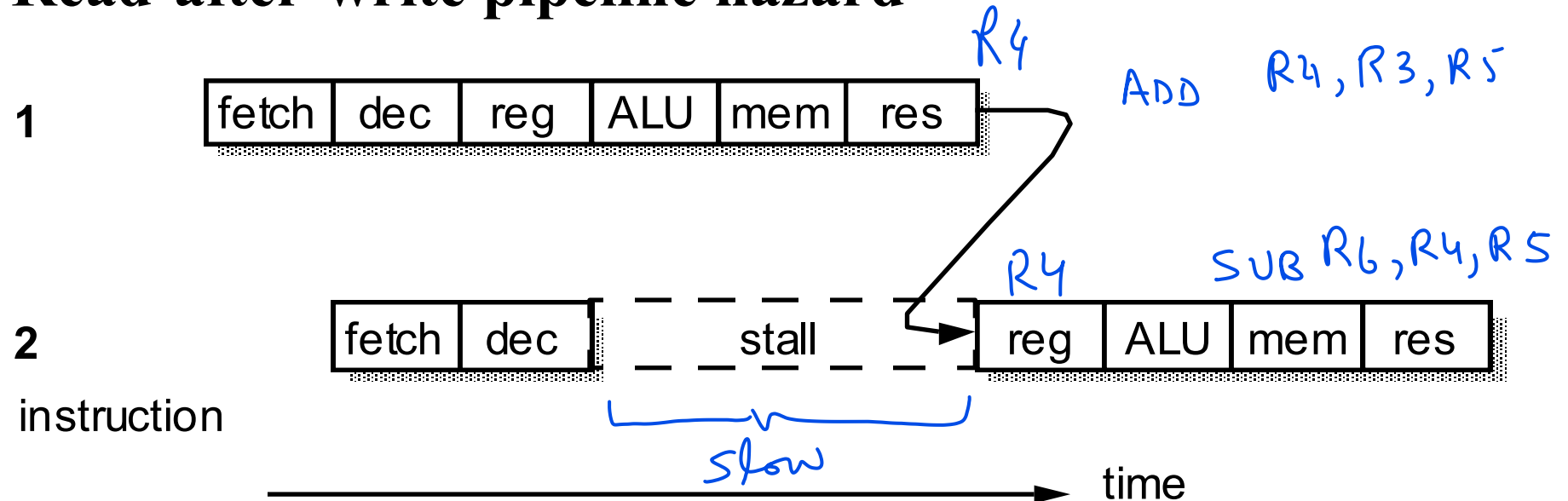
BRANCH INST.

- Data Hazards

Pipelined Instruction Execution (~~5~~-stage Pipeline)



Read-after-write pipeline hazard



CPU Power Consumption

i7 > 50W

- Most modern CPUs are designed with power consumption in mind to some degree.
- Power vs. energy:
 - Heat depends on power consumption;
 - Battery life depends on energy consumption.

Power Saving Strategies

- Reduce power supply voltage.
- Run at lower clock frequency.

5V $\rightarrow$ 3.3V, 1.2V
when CPU is idle
or simple calc.
Software by less memory access

Microcontrollers and Embedded CPUs

Microcontrollers are available in 4 to 32-bit word sizes.
8-bit or 16-bit micro-controllers are widely used.

Popular Microcontrollers

VERILOG
VHDL

RISC-V

Model	Pins I/O	RAM	ROM	Counter Timer	Remarks
Intel 8051	40:32	128	4K	2	128K External, Serial port
Motorola 6811	52:40	256	8K	2	A/D, Watch Dog timer, Serial Port
ARM CPUs TMS470-ARM	Soft	8-64KB	64KB-1MB		16/32-bit RISC DMA, Watch Dog timer, CAN, I ² C
Intel, 80C196	68:40	232	8K	2	16-bit 64K External, Serial, A/D, WD
Intel, 80960 (i960)	132	512 Ins Cache	--	20MHz Clock	32-bit bus, FPU, Interrupt control No I/O on-chip
MIPS32 4K	Soft core	I/D Cache	--		32-bit RISC, EJTAG and On-chip bus
MC68360 QUICC	240/241			4 or 16-bit	32-bit, WD, Fault-tolerant, 4-Ethernet, 2 serial, Integrated Com Controller
Nios II	Soft core	On-chip Ram		IP	32-bit RISC, Soft Core for Embedded CPUs on FPGA

68538

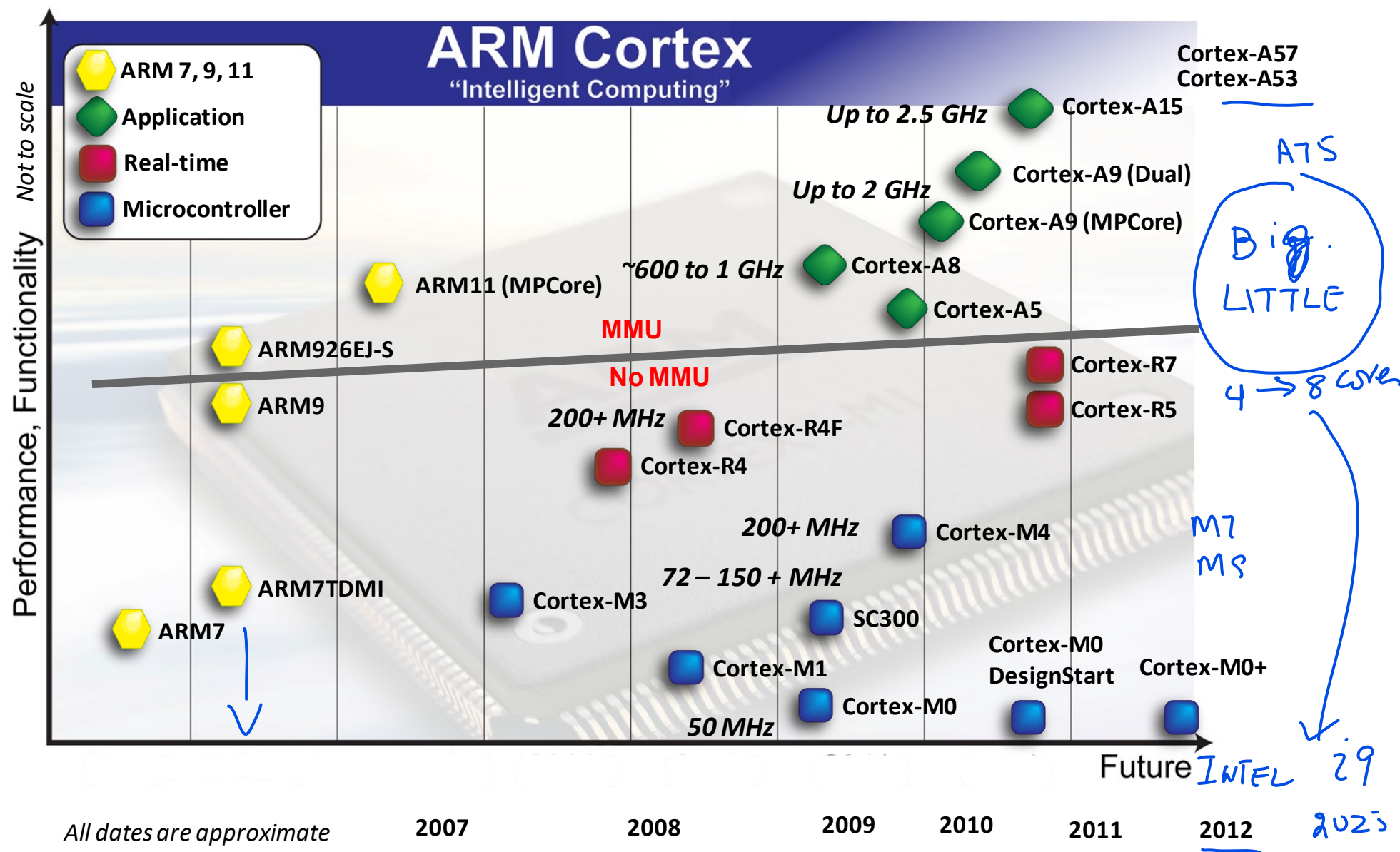
ARM7

ALTERA / INTEL

ARM[®]

ARM Powered Products





ARM CPU

Versions, cores and architectures ?

- What is the difference between ARM7™ and ARMv7 ?
- ARM doesn't make chips....well maybe a few test chips.

Family	Architecture	Cores
ARM7TDMI	ARMv4T	ARM7TDMI(S) 2004
ARM9 ARM9E	ARMv5TE(J)	ARM926EJ-S, ARM966E-S
ARM11	ARMv6 (T2)	ARM1136(F), 1156T2(F)-S, 1176JZ(F), ARM11 MPCore™
Cortex-A	ARMv7-A	Cortex-A5, A7, A8, A9, A15
Cortex-R	ARMv7-R	Cortex-R4(F)
Cortex-M3/M4	ARMv7-M ARMv6-M	Cortex-M3, M4 Cortex-M1, M0
NEW! 202?	ARMv8-A v9	64 Bit A72, A73 . . (2018) A510, A710

ARM Processor Licenses (the public ones)

- ARMv8-A ? NVIDIA, Applied Micro, Cavium, AMD, Broadcom, Calxeda, HiSilicon, Samsung and STMicroelectronics
- Cortex-A15 4 ST-Ericson, TI, Samsung, nVIDIA
- Cortex-A9 9 NEC, nVIDIA, STMicroelectronics, TI, Toshiba ...
- Cortex-A8 9 Broadcom, Freescale, Matsushita, Samsung, STMicroelectronics, Texas Instruments, PMC-Sierra
- Cortex-A5 3 AMD,
- Cortex-R4(F) 14 Broadcom, Texas Instruments, Toshiba, Inf
- Cortex-M4 5 Freescale, NXP, Atmel, ST
- Cortex-M3 29 Actel, Broadcom, Energy Micro, Luminary Micro, NXP, STMicroelectronics, TI, Toshiba, Zilog, ...
- Cortex-M0 14 Austria-microsystems, Chungbuk Technopark, NXP, Triad Semiconductor, Melfas
- Cortex-M0+ Freescale, NXP
- ARM7 172, ARM9 271, ARM11 82

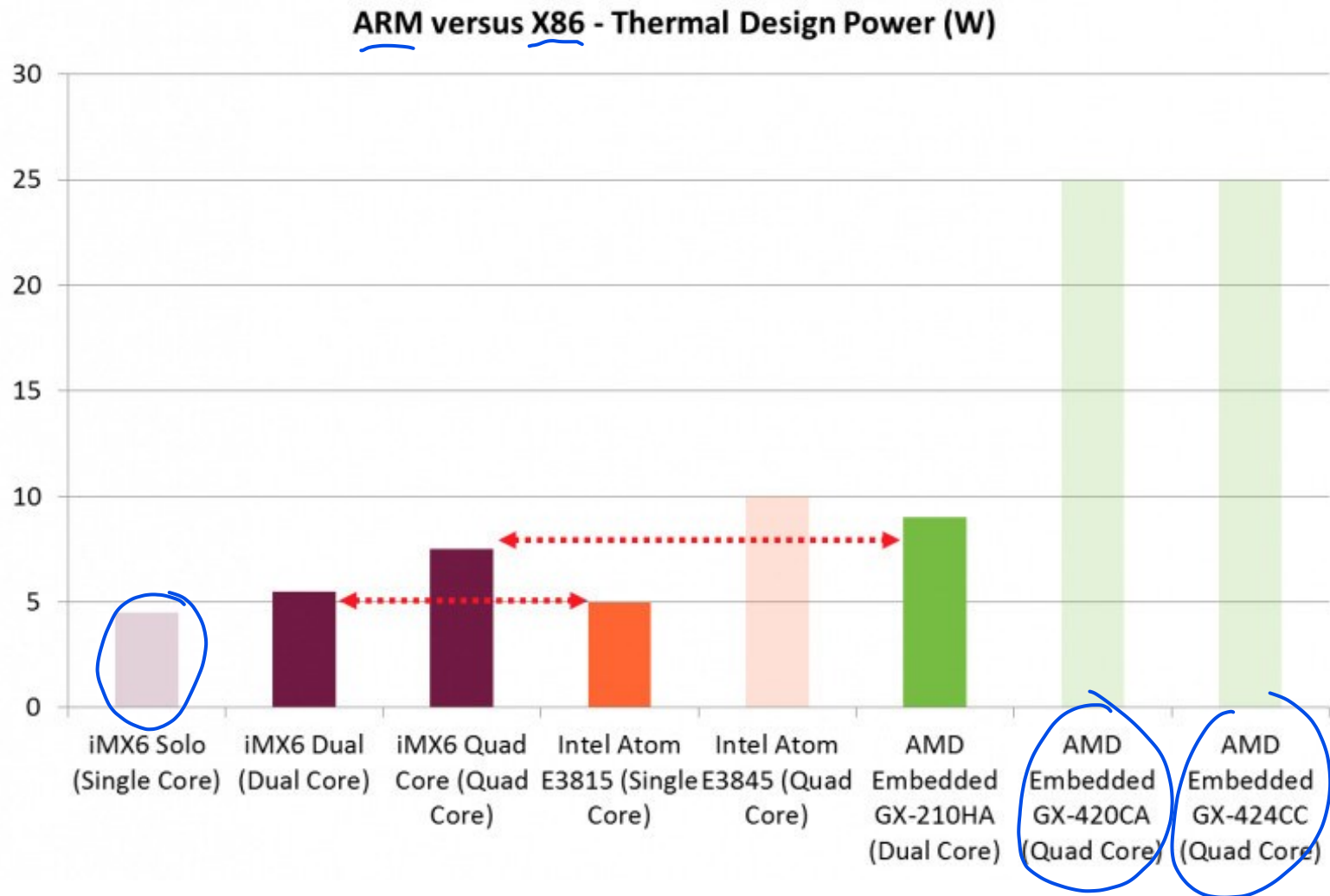
COE 838

BIT BANDING

NETLIST/
LAYOUT

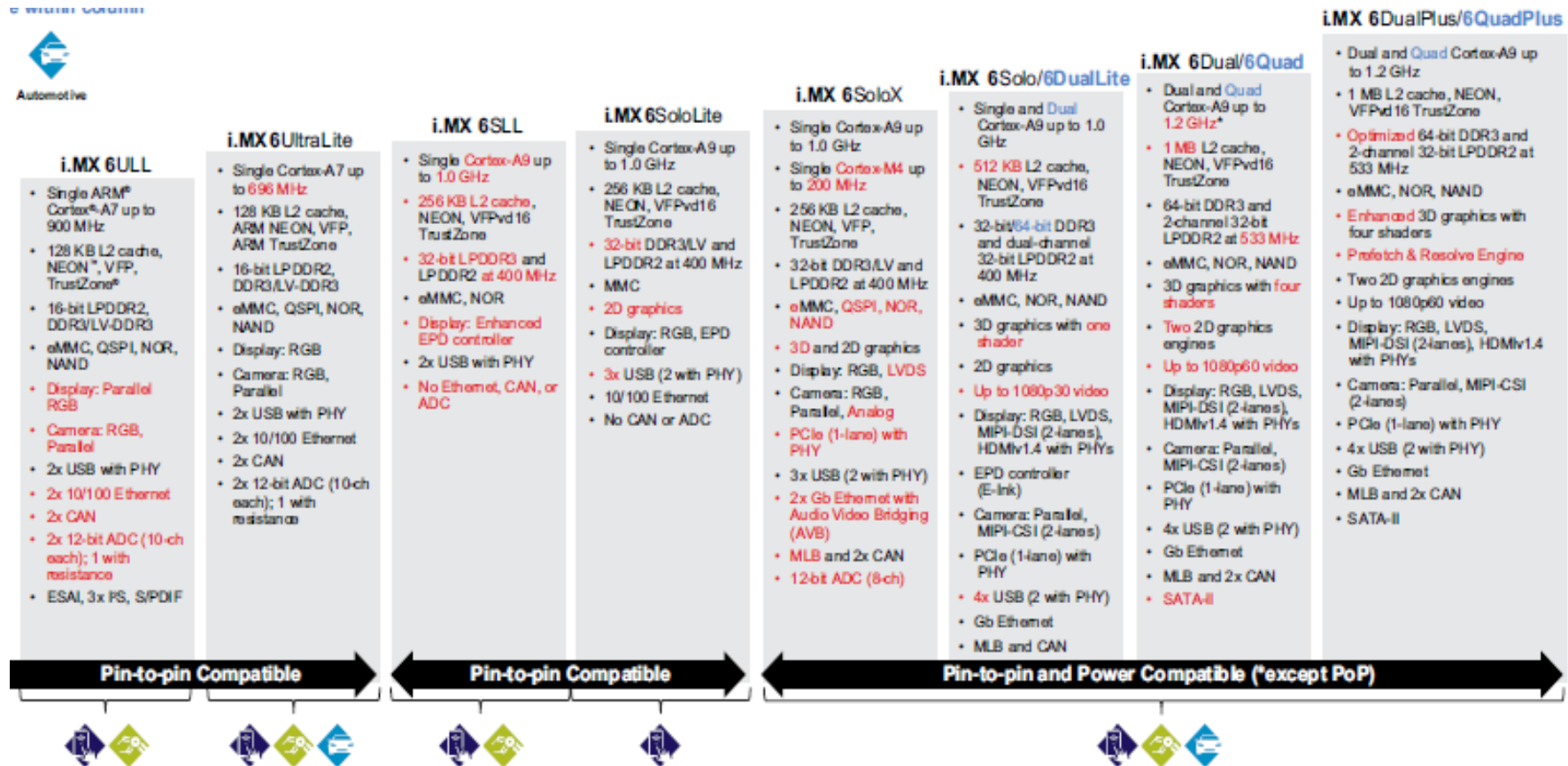
SEMICONDUCTOR

ARM vs. x86 Power



iMX6 is Cortex A7, A9 and M4 multicore CPUs NXP SoCs

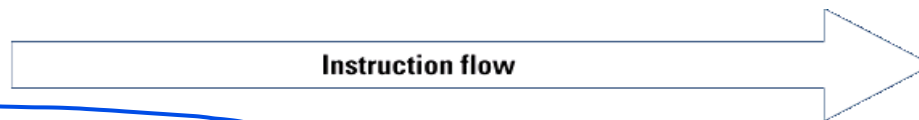
NXP iMX6 Multiple ARM Processors



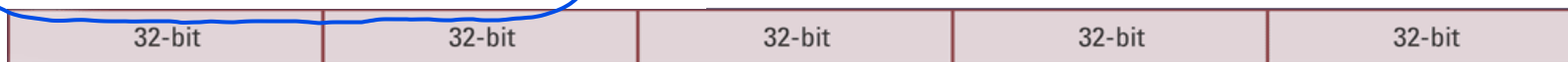
ARM Instruction Sets

ARCHITECTURE
ISA

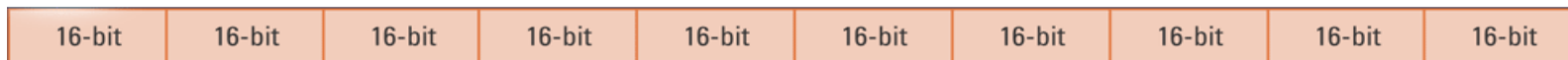
- ARM (32 bit) now referred as AArch32
- Thumb (16 bit) OLDER
- Thumb2: Cortex-Mx processors. Cortex-R, A have Thumb2 + ARM.
- A64 (64 bit) referred as AArch64



ARM now called AArch32



Thumb (actually includes all ARM 32 bit instructions)

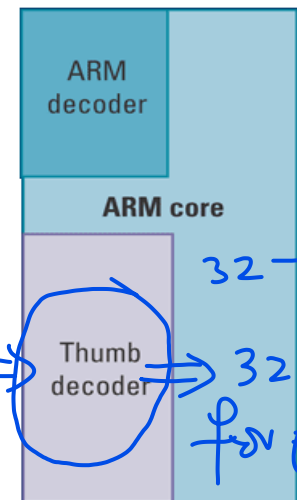
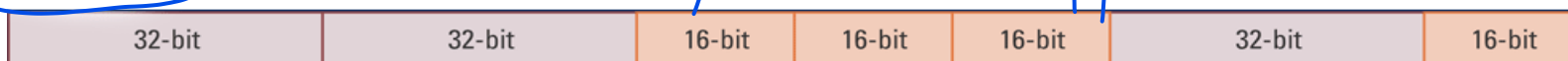


Thumb-2

FOR SMALLER/SIMPLE Application

16-bit

=>



A64 AArch64



CHIP

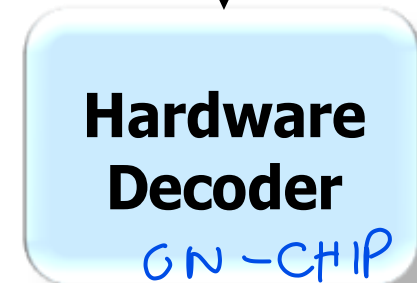
Tools: Keil MDK™ with μ Vision:

- For Cortex-M and Cortex-R processors.
- Proprietary IDE μ Vision
- ARM compiler, assembler and linker.
- ULINK2, ULINK*pro*, CMSIS-DAP + more debug adapters.
- Many board support packages (BSP) and examples.
- MDK Professional: TCP/IP. CAN, USB & Flash middleware.
- Serial Wire Viewer and ETM, MTB & ETB Trace supported.
- Evaluation version is free from www.keil.com/arm.
- Is complete turn-key package: no add-ons needed to buy.
- Valuable technical support included for one year. Can be easily extended.
- Keil RTX RTOS included free with source code.

Three Types of Instruction Sets

- ARM Instruction Set
 - Instructions are 32 bits wide
 - Original RISC (lots of parallelism)
 - “Load/Store” Architecture
- Thumb Instruction Set
 - Subset of ARM instructions, some restrictions
 - Instructions are 16 bits wide (more like CISC)
 - Intended for compilers
 - Less parallelism, longer instruction sequences
 - but total code size is 30% smaller
- Jazelle Instruction Set
 - Java byte codes

**16-bit Thumb
Instruction**



**32-bit ARM
Instruction**

EXECUTE

The Wall Street Journal July 18, 2016

SoftBank to Buy ARM Holdings for \$32 Billion

Japanese group's deal for U.K.-based chip designer comes together in just two weeks

Three Instruction Sets

	ARM	Thumb*	Jazelle
Instruction Size	32 bits	16 bits	8 bits
Core instructions	58	30	> 60% of Java byte codes in hardware; rest in software
Conditional Execution	most	Only branch instructions or in an IT block	N/A <i>Control Hazard?</i>
Data processing instructions	Access to barrel shifter and ALU	Separate barrel shifter and ALU instructions	<u>N/A</u>
Program status register	Read/write in privileged mode	No direct access	<u>N/A</u>
Register usage	15 general purpose registers + pc	8 general purpose registers + 7 high registers + pc	N/A

ARMv7 vs Thumb Instruction Set

- ARM has 2 instructions sets = traditional ARM ISA (i.e. ARMv7, etc) and the condensed Thumb ISA (16/32bit)
- ARMv7 is a 32-bit standard instruction set derived by ARM
- **Thumb2** incorporates 16/32-bit mix by using a Unified Assembler Language (UAL) to alternate between these 2 ISAs, basically adds suffixes the instructions ['N' (16) and 'W' (32)] to let the compiler know if it's 16-bit or 32-bit
- Each Thumb instruction directly correlates to an equivalent 32-bit ARM instruction version, which has the same effect on the processor and only their encodings are different.

Thumb | 0 | 0 | 0 | OP (2b) | Imm5 | Rs (3b) | Rd (3b)

ARM | cond 4b | 0 | 0 | 1 | op (4b) | S | Rn (4b) | Rd (4b) | rotate (4b) | imm8

Thumb when in 'N' mode has less register space to work with, and can only deal with smaller imm values (for loads/stores, branching etc.)

HISTORY of ARM ISA

ARM initially came out with Thumb, and then introduced Thumb-2 technology (still using Thumb ISA) bc ARM had two separate syntaxes which made compilation and execution more complex.

- Solved with UAL
- Using Thumb only one ISA could be active at a time, switching in the middle of an application gives some overhead (i.e. BLX)
- Thumb is used to trim down code size and memory utilization with hardly no change to the core

Decreased code size = less memory usage = lower power consumption

Reduces code size by > 30%

Thumb still has 32-bit bus and operates with a 32bit data-width.

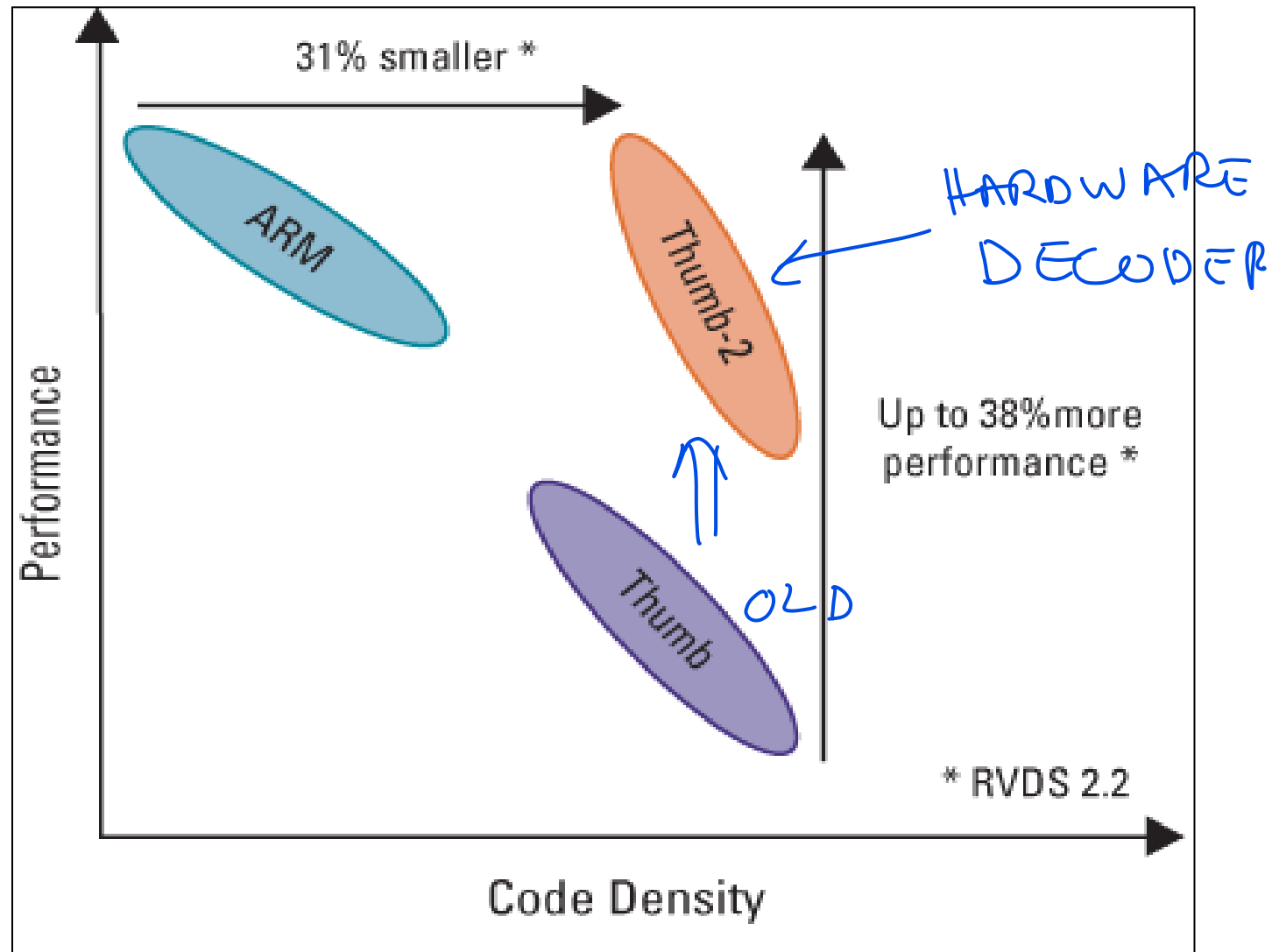
More instructions can be fetched from memory as instructions are smaller and code size is condensed. In general, fetching instructions is slower than executing instructions

* Less instruction memory utilization

SAVE POWER

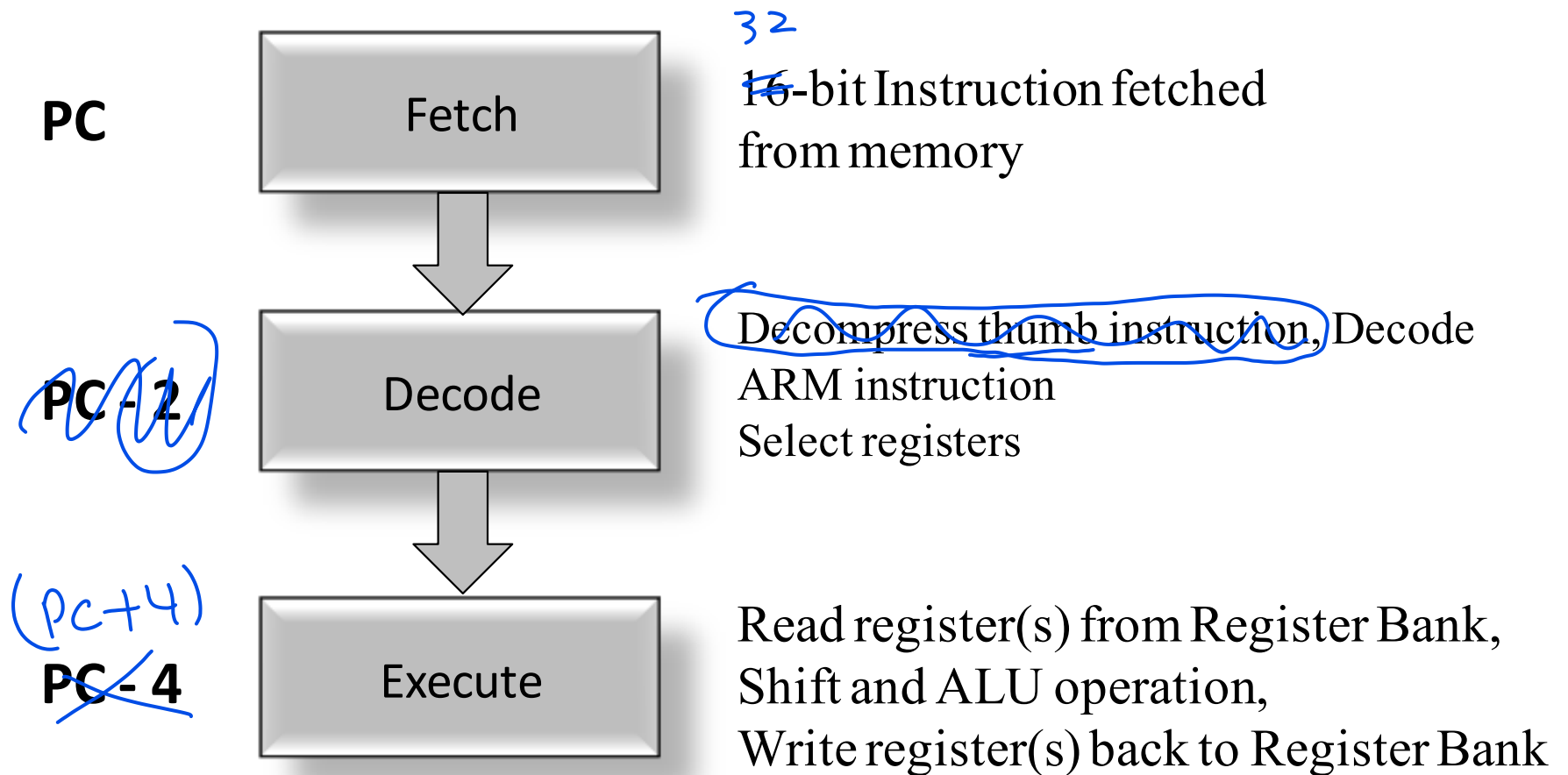
* Very little performance difference.

Performance/Code for Instruction Sets



-Use Thumb for limited memory as well as low power.

Pipelined Instruction Fetch, Decode and Execute



ARM7 Architecture

- Load/store architecture
- Most instructions are RISCy (NOT PURE RISC)
Some multi-register operations take multiple cycles
- All instructions can be executed conditionally

LOAD from Memory
STORE to Memory

~~ADD R1, R2, [Mem]~~

ARM7 is a small, low power, 32-bit microprocessor.
Three-stage pipeline, each stage takes one clock cycle

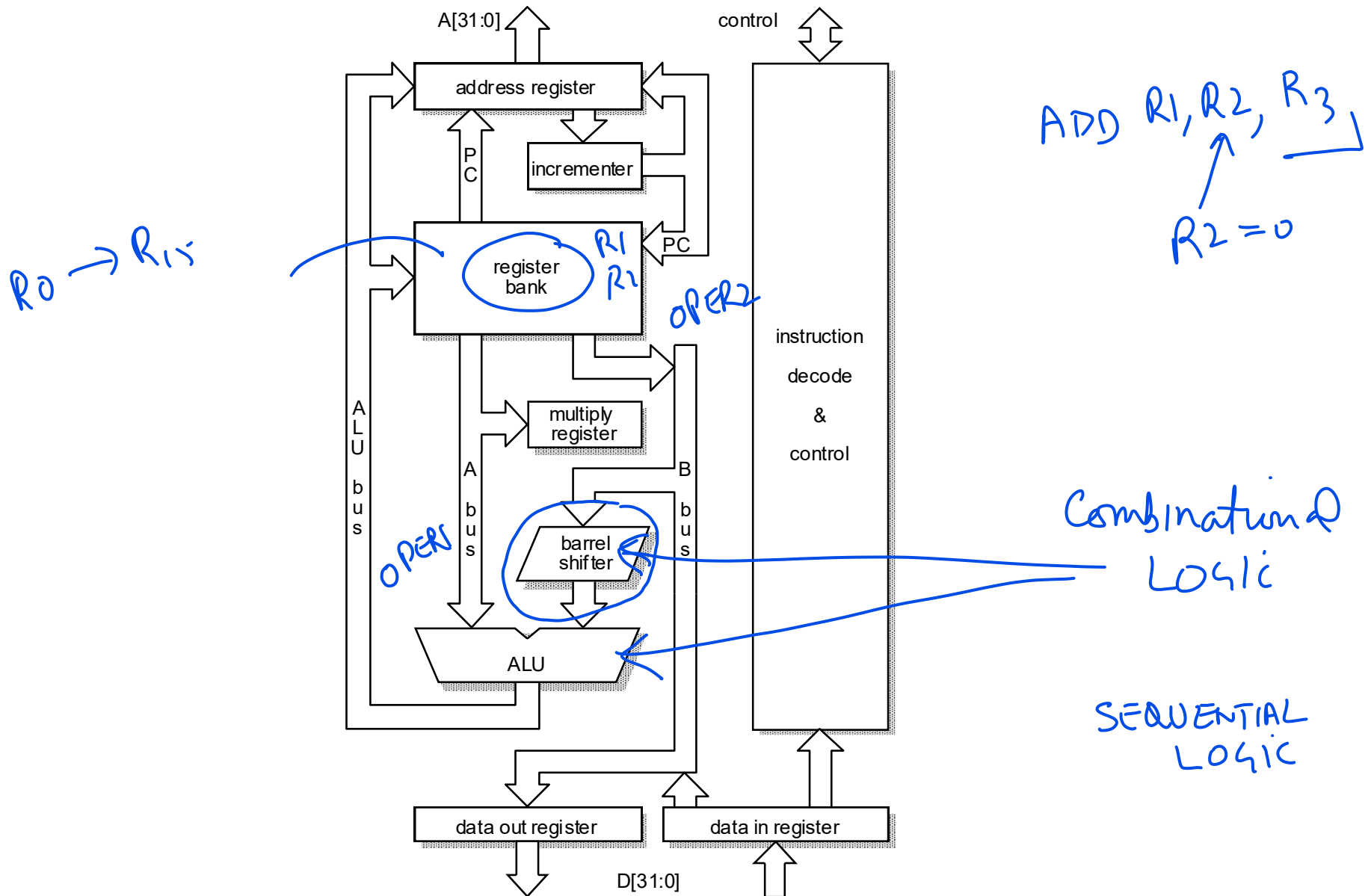
- Instruction fetch from memory
- Instruction decode
- Instruction execution.
 - Register read
 - A shift applied to one operand and the ALU operation
 - Register write

ADD DEST, OPER1, OP2

e.g. SHIFT/ADD

ALU & {BARREL SHIFTER

ARM CPU Core Organization



ARM7 Features

Combined Shift and ALU Execution Stage

- A single instruction can specify one of its two source operands for shifting or rotation before it is passed to the ALU
- Allows very efficient bit manipulation and scaling code
- Eliminates virtually single shift instructions from ARM code.
ARM7 CPU does not have explicit shift instructions.
- A move instruction can apply a shift to its operand

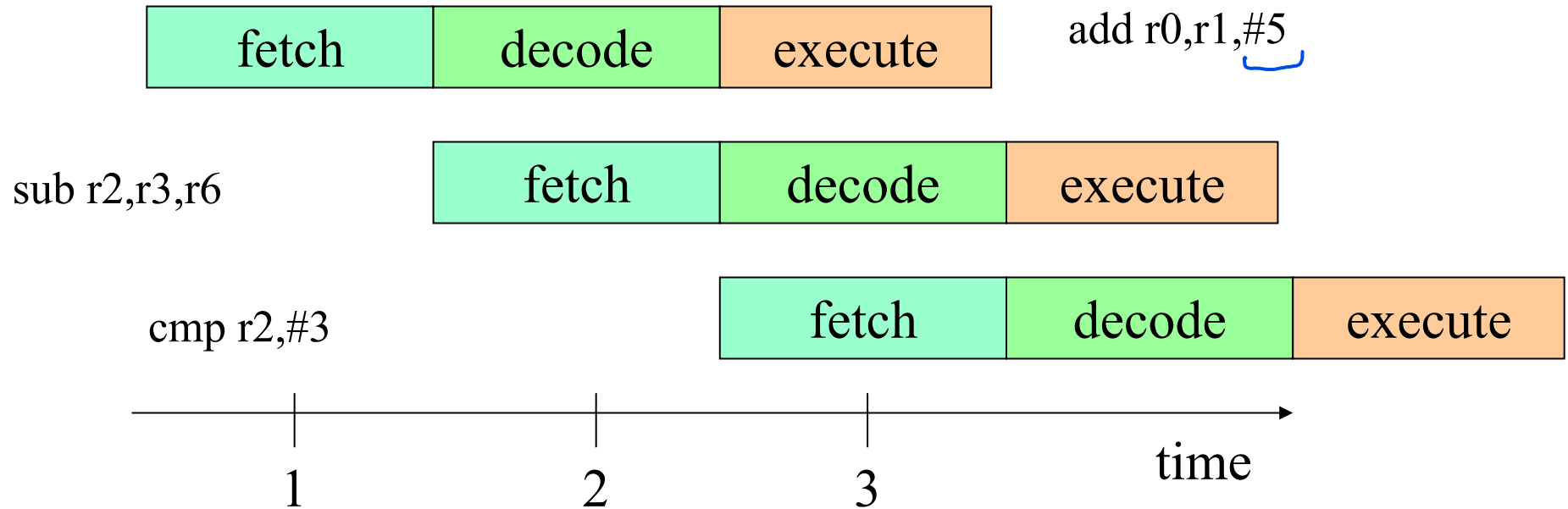
MOVE R1, R3

ARM7 uses von-Neumann memory architecture where instructions and data occupy single address space that can limit the performance

- Instruction fetching (and execution) must stop for instructions that access memory
- The pipeline stalls during load and store operations, ARM7 can continue useful work.

STRUCTURAL HAZARD

ARM7 Pipeline Execution



- **Latency**

Time it takes for an instruction to get through the pipeline.

- **Throughput**

Number of instructions executed per time period.

ARM CPU Features: Modified RISC

Multiple Load and Store Operation

Reduce the penalty of data accesses during a stall in the pipeline
Multiple load/store instructions can move any of the ARM registers to and from memory, and update the memory address register automatically after the transfer.

- This not only allows one instruction to transfer many words of data (in a single bus burst), it also reduces the amount of instructions needed to transfer data.
- Make the ARM code smaller than other 32-bit CPUs
- These instructions can specify an update of the base address register with a new address after the transfer.

RISC CPU architectures would normally use a second instruction (add or subtract) to form the next address in a sequence.

ARM does it automatically with a single bit in the instruction, again a useful saving in code size.

ARM CPU (More) Features

All instructions are conditionally executed:

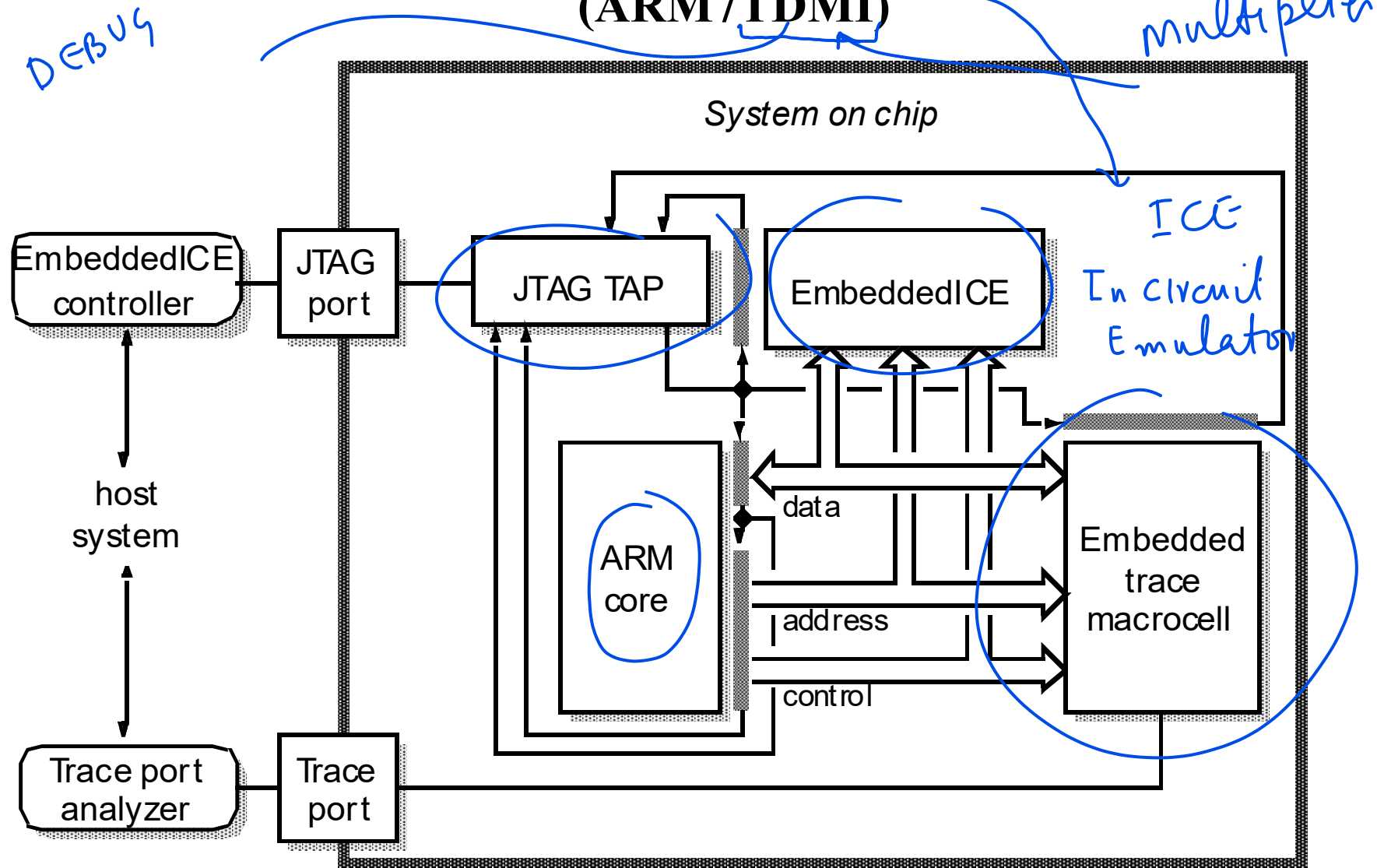
- A very useful feature
 - Loads, stores, procedure calls and returns, and all other operations can execute conditionally after some prior instruction to set the condition code flags
 - Any ALU instruction may set the flags
 - This eliminates short forward branches in ARM code
 - It also improves code density and avoids flushing the pipeline for branches and increase execution performance
- LEAD TO REMOVING BRANCH INST. FROM THE CODE*
- N, C, V, Z*
- Most CPU architectures have conditional branch instructions
 - These follow a test or compare instruction to control the flow of execution through the program
 - Some architectures also have a conditional move instruction, allowing data to be conditionally transferred between registers

BEO LABEL

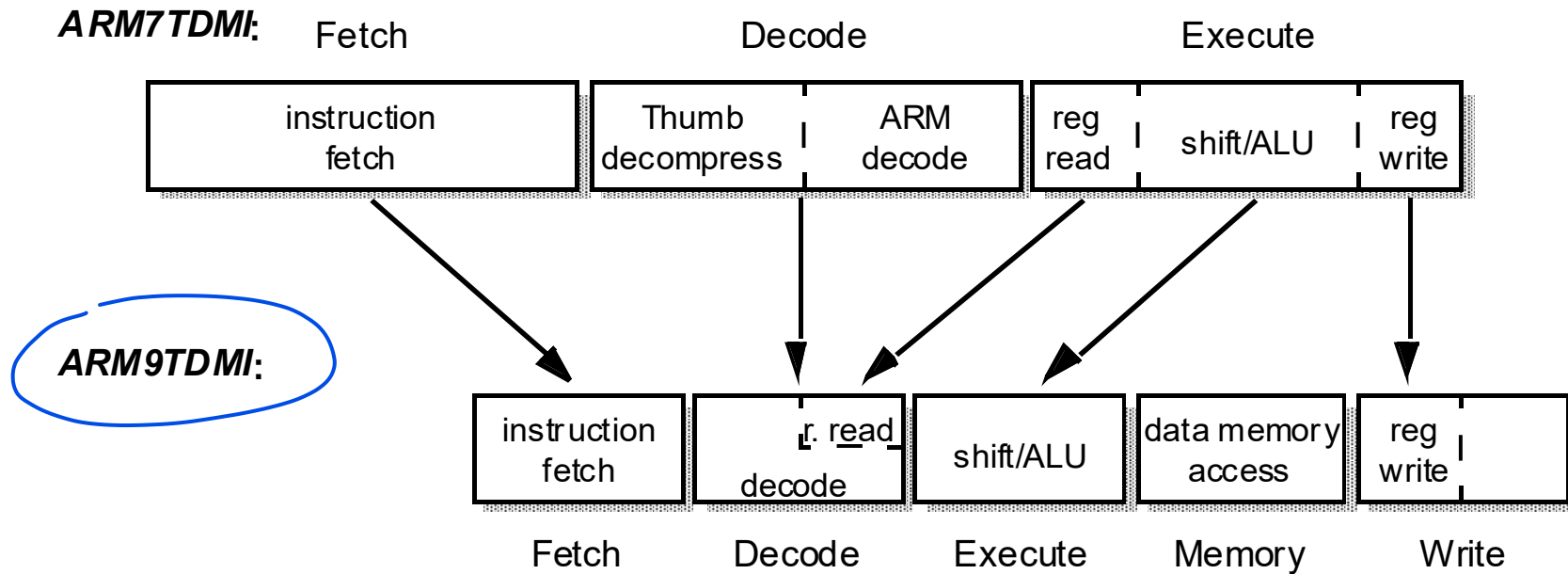
(ADD EQ R1, R2, R3)

LABEL ADD R1, R2, P.

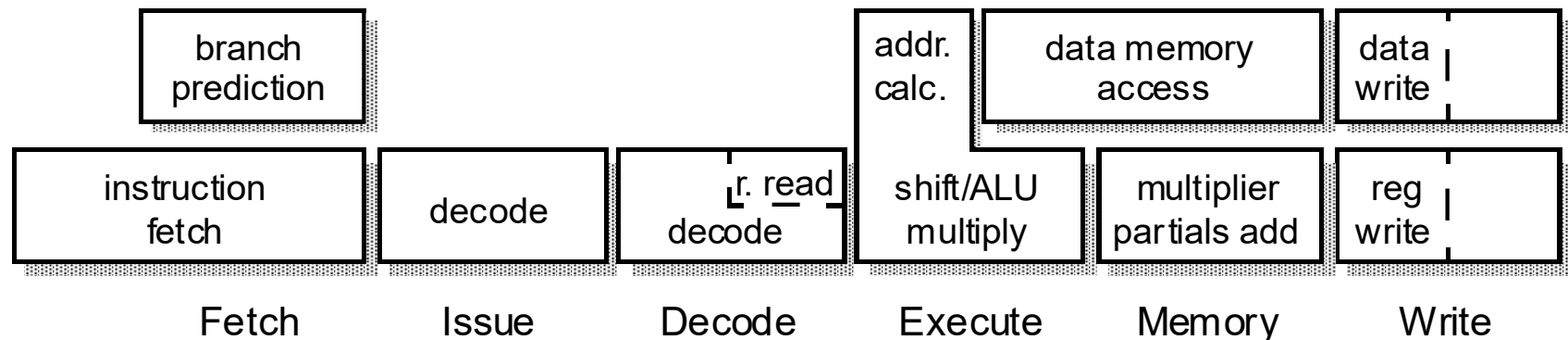
Real-time Debug System Organization (ARM7TDMI)



ARM7TDMI and ARM9TDMI Pipeline



The ARM10TDMI pipeline



ARM Architectures

Core

Architecture

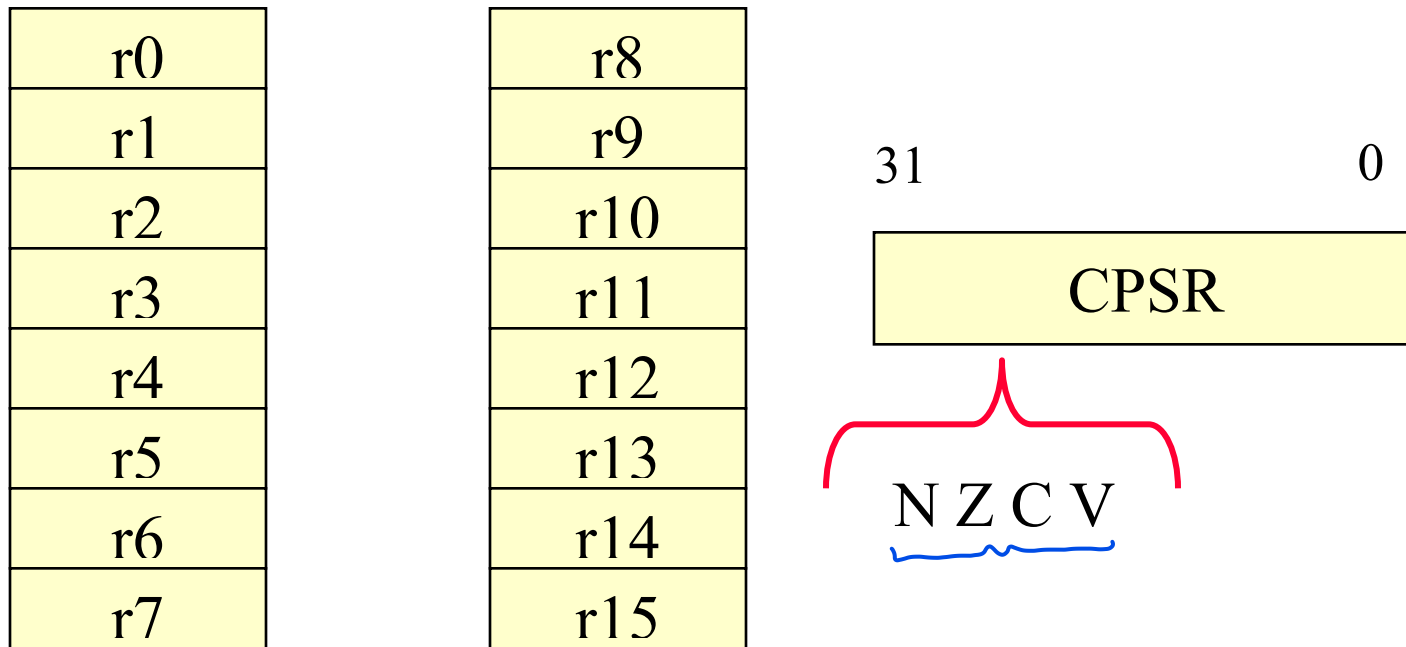
Classic ARM Processors

ARM1	v1
ARM2, ARM2as, ARM3	v2, v2a
<u>ARM6</u> , ARM600, ARM610	v3
<u>ARM7TDMI</u> , ARM710T, ARM720T, ARM740T	v4T
ARM8, ARM810	v4 2004
ARM9TDMI, ARM920T, ARM940T	v4T
ARM9ES	v5TE
ARM10TDMI, ARM1020E	v5TE
<u>ARM11</u>	<u>v6</u>
.....	

ARM Cortex Processors

ARM Cortex-M3	} standard embedded system	v7M
ARM Cortex-M4		v7ME
ARM Cortex-R4, R5, R7	Real Time Embedded System	v7R
ARM Cortex-A5, A8, A9, A15	HIGH PERFORMANCE SYSTEM	v7A

ARM7: Programming Model

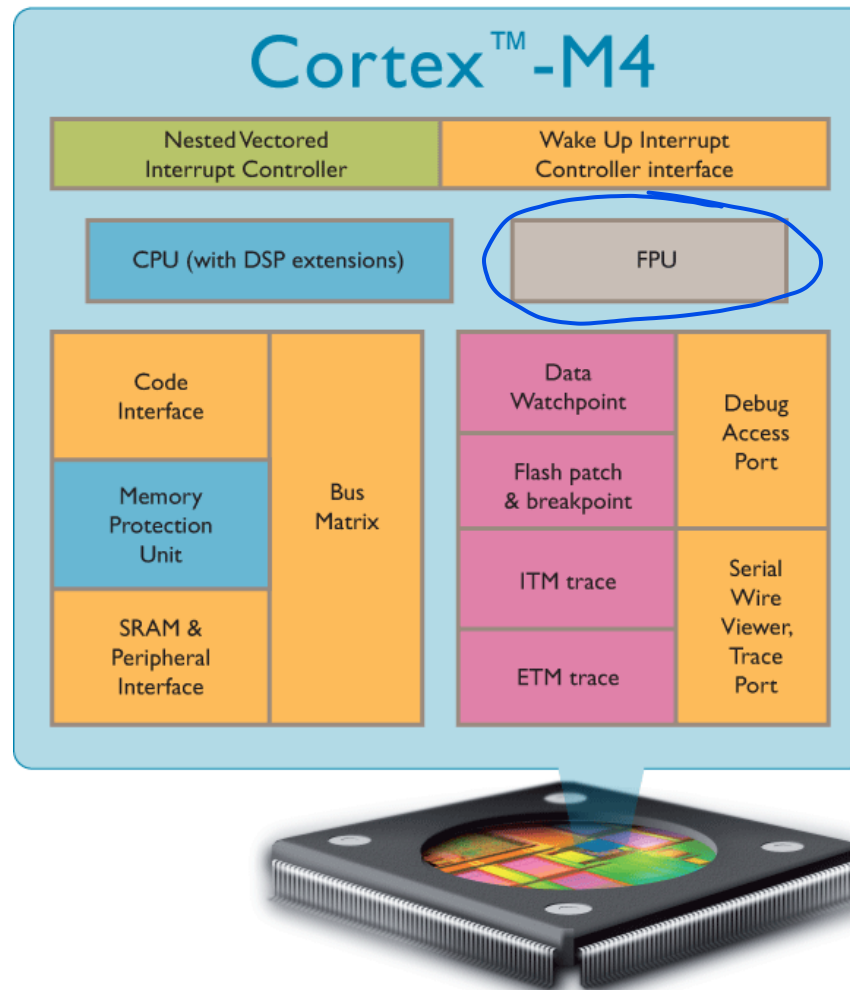


- Word is 32 bits long.
- Word can be divided into four 8-bit bytes.
- ARM addresses can be 32 bits long.
- Address refers to byte.
Address 4 starts at byte 4.

4 GBYTE Memory

ARM Cortex-M4

Latest Cortex-M series CPU that has a combination of efficient signal processing and low-power.

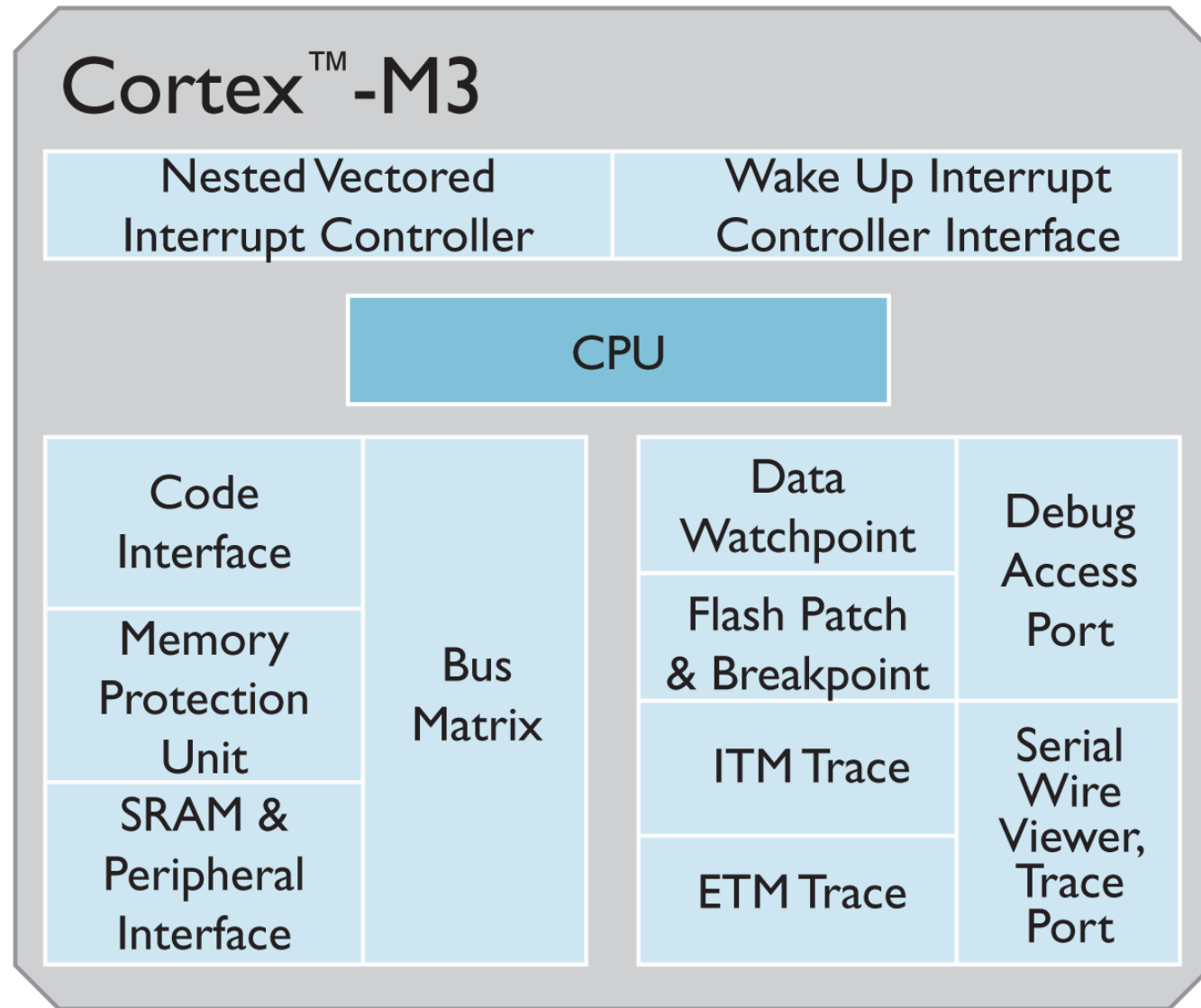


Harvard vs. Van Neumann Architecture

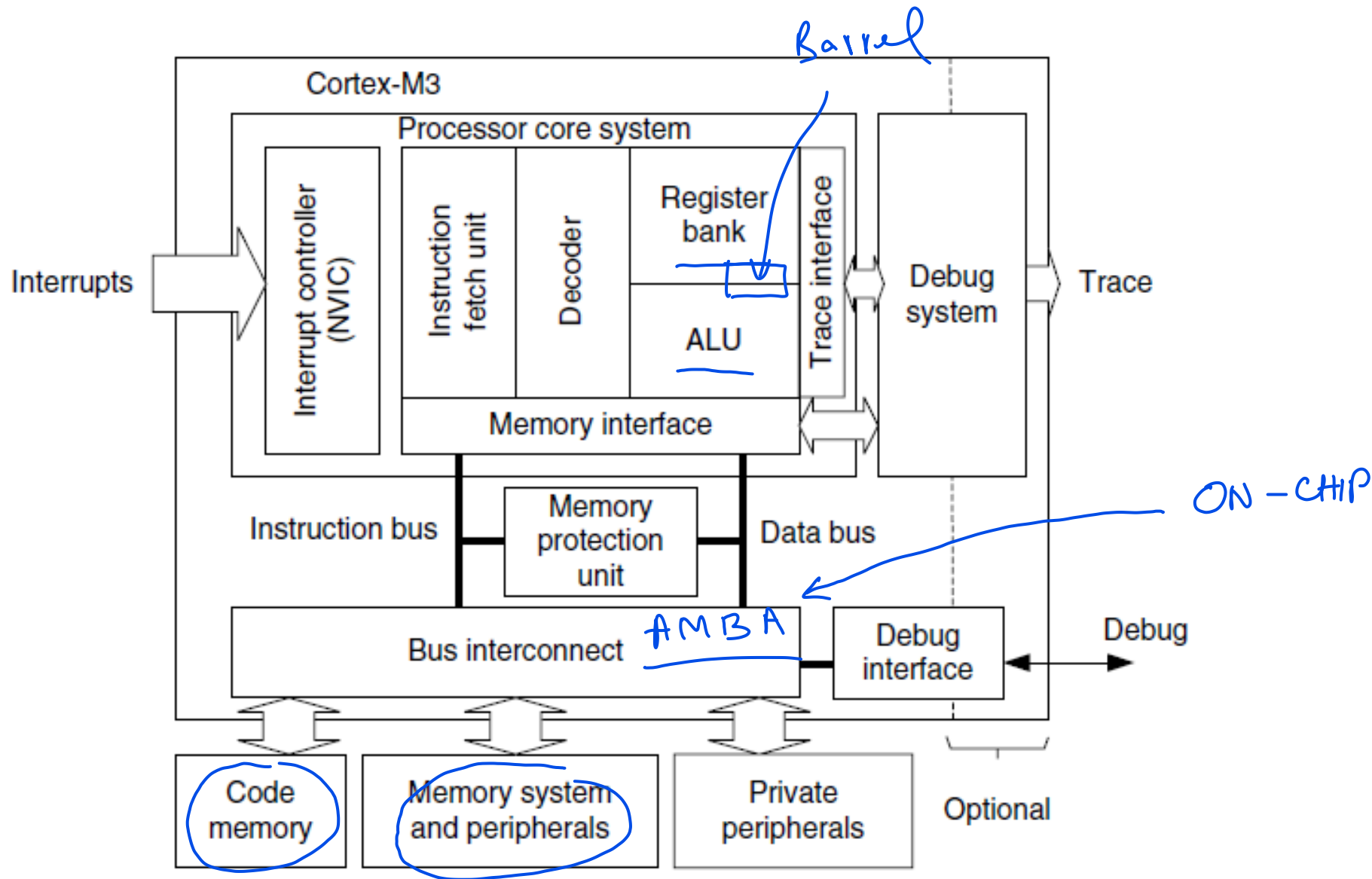
ARM Cortex-M optional components^{[6][7]}

ARM Cortex-M	SysTick Timer	Bit-banding	Memory Protection Unit (MPU)	Tightly-Coupled Memory (TCM)	CPU cache	Memory architecture	ARM architecture
Cortex-M0 ^[1]	Optional*	Optional ^[9]	No	No	No ^[10]	<u>Von Neumann</u>	ARMv6-M
Cortex-M0+ ^[2]	Optional*	Optional ^[9]	Optional (8)	No	No	<u>Von Neumann</u>	ARMv6-M
Cortex-M1 ^[3]	Optional	Optional	No	Optional	No	<u>Von Neumann</u>	ARMv6-M
Cortex-M3 ^[4]	Yes	Optional*	Optional (8)	No	No	<u>Harvard</u>	ARMv7-M
Cortex-M4 ^[5]	Yes	Optional*	Optional (8)	No	⌚ Possible ^[11]	Harvard	ARMv7E-M
Cortex-M7	Yes	No	Optional (8 or 16)	Optional	Optional	Harvard	ARMv7E-M

ARM Cortex-M3



ARM Cortex-M3 Core System



ARM Cortex-M3 Core System

NESTED VECTORED INT. CONTROLLER

NVIC - Interrupt controller (will get back to this in interrupt section).

Memory Protection Unit (MPU) - invokes rules for accessing memory;

RTOS Imp.

SYSTICK - countdown timer, used to generate interrupts. Real-time clock

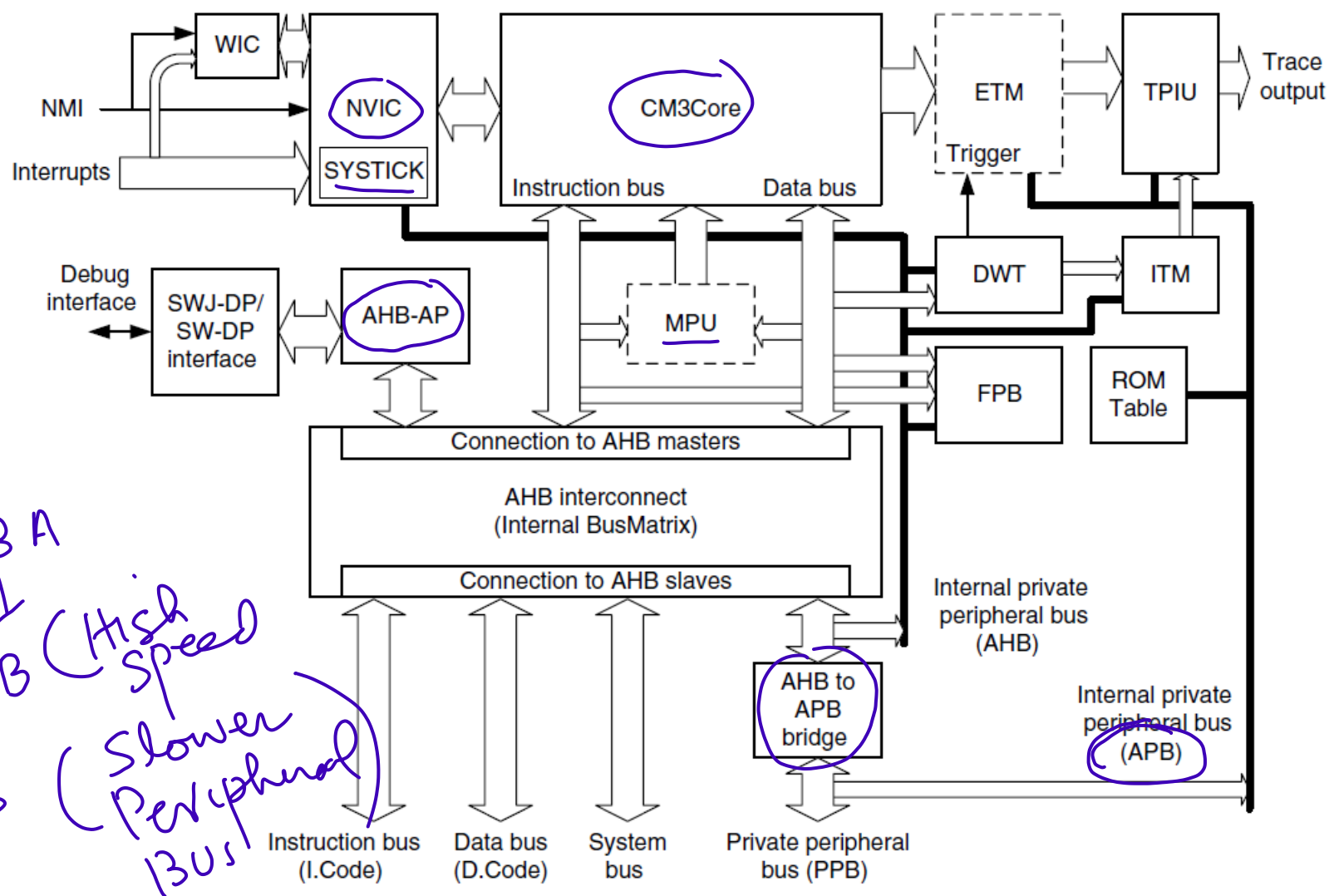
WIC - unit for waking up the CPU

ROM - small lookup table that stores configuration information

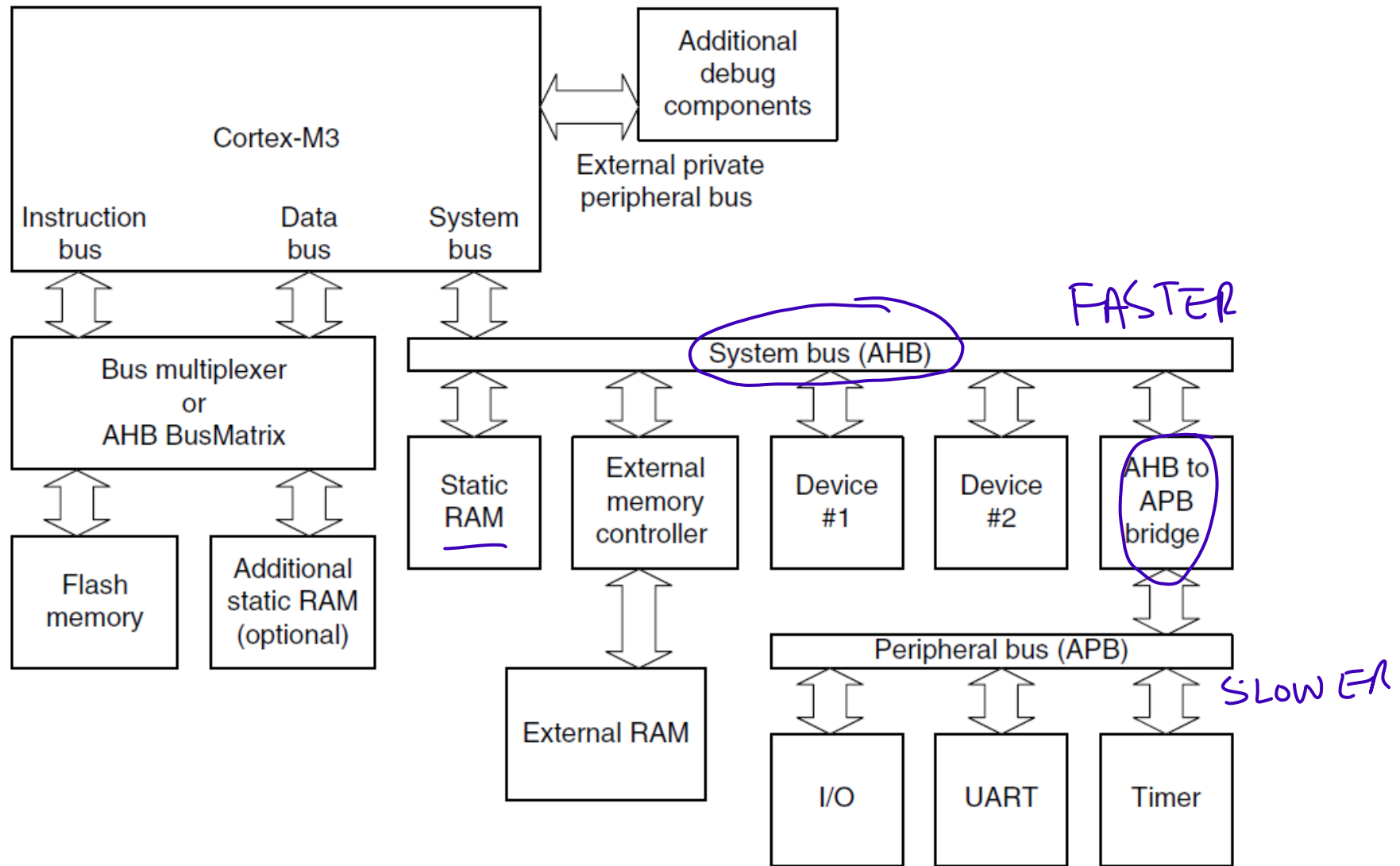
BusMatrix - interconnect used to transfer data on different busses simultaneously.

Rest are **debugging** blocks ^{JTAG} ----> SW-DP (Serial Wire Debug Port), ETM (Embedded Trace Macrocell), DWT (Data Watch-point and Trace), ITM (Instrumentation Trace Macrocell), etc.

Cortex-M3 CPU Overview



Cortex-M3 Bus System



ARM Cortex-M3 Core System

- ARM uses the AMBA bus
- AMBA is an open standard on-chip interconnect specification (i.e. it has protocols, handshake methods, etc.)

Cortex-M3 uses the following AMBA based busses: *APB*

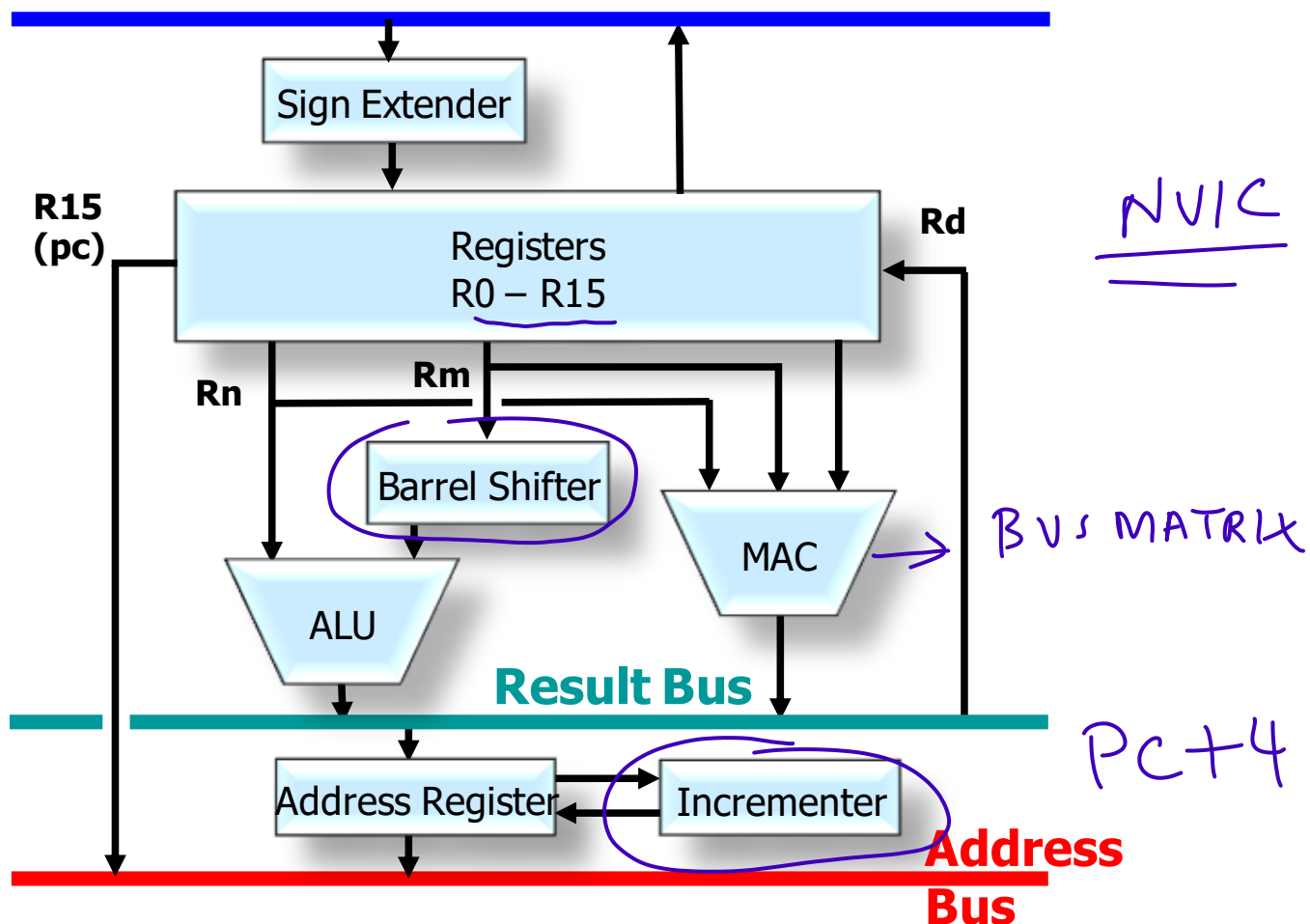
- 1) Advanced Peripheral Bus - low bandwidth, used for processor to peripheral transactions
- 2) ARM High-Performance Bus - high bandwidth/data-width transfers for high performance, uses bursts *AHB*

Cortex M3 has a Harvard architecture CPU: there are dedicated instruction and data busses.

ARM Cortex-M3

Introduced in 2004, the mainstream ARM processor developed specifically with microcontroller applications in mind.

Data Bus



ARM Cortex-M3

- Implement Thumb-2 instruction subset of ARM Instruction Set.
- Most Thumb-2 instructions are 16-bit wide that are expanded internally to a full 32-bit ARM instructions.
- ARM CPUs are capable of performing multiple low-level operations in parallel.
- A hardware sign extender convert 8-16 bit operands to 32-bit
- Load store architecture.
- Barrel shifter allows operand R_m to be shifted first and then ALU can perform another operation (e.g. add, subtract, mul etc.)
- MAC is a sort of memory address calculator for different addressing of arrays and repetitive address calculations.
- R_0 - R_{12} GPR, R_{13} - R_{15} special purpose registers i.e. SP, PC and LR (that holds the return address when a subroutine is called.

ARM Registers

**ARM
Mode:**
**15 general
purpose
registers**

R0
R1
R2
R3
R4
R5
R6
R7
R8
R9
R10
R11
R12
R13: Stack Pointer (SP)
R14: Link Register (LR)
R15: Program Counter (PC)

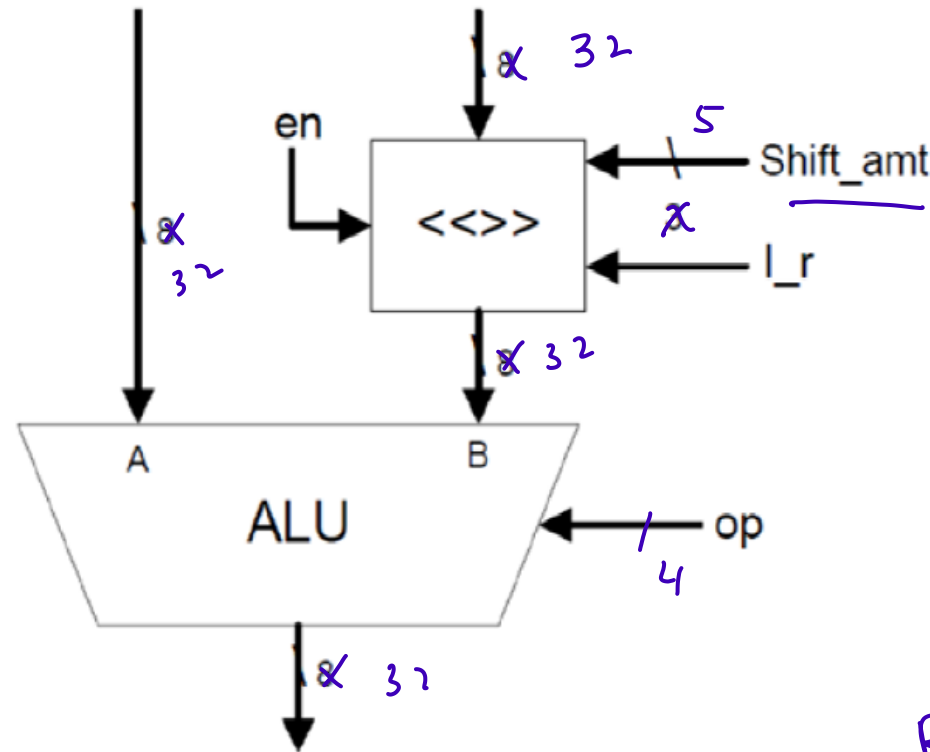
**Thumb
Mode:**

**8 general
purpose
registers**

**7 “high”
registers**

**r8-R12 only
accessible
with MOV,
ADD, or
CMP**

Barrel Shifting



DIV R1, R2, #4

MUL R1, R2, #17
 $R1 \leftarrow 16 \times R2 + R2$

- Barrel shifter rotates/shift instruction operand prior to inputting the value into the ALU

4
 $R2 \times 4 \Rightarrow R1$

① *MUL R1 R2 #2* (LSL R1 R2 #2)
 ② *ADD R5, R1, R4*

$R4 + R1 \Rightarrow R5$

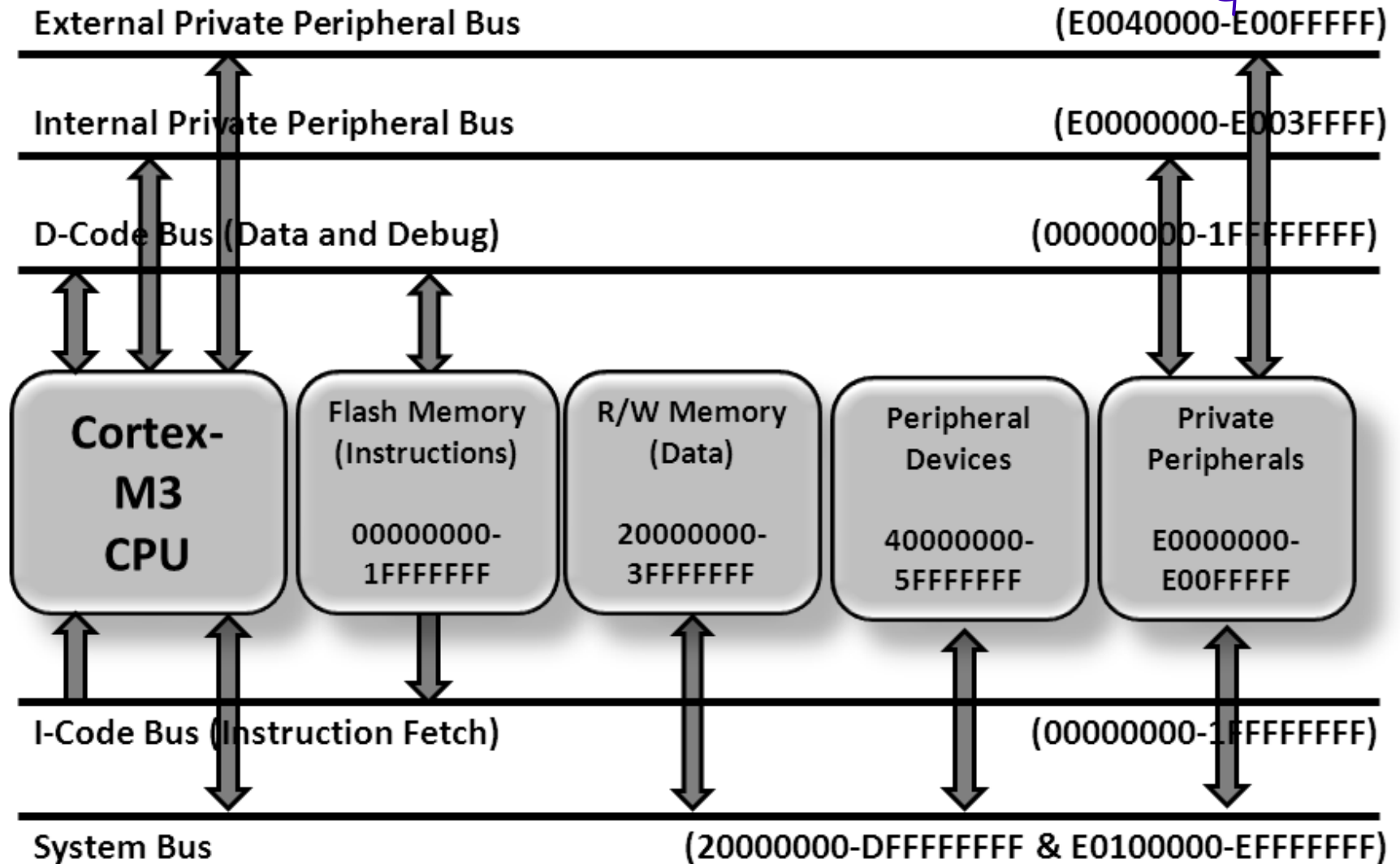
$\Rightarrow$ *ADD AL R5, R4, R2 LSL #2*

ADD R2, R1, R1 LSR #2

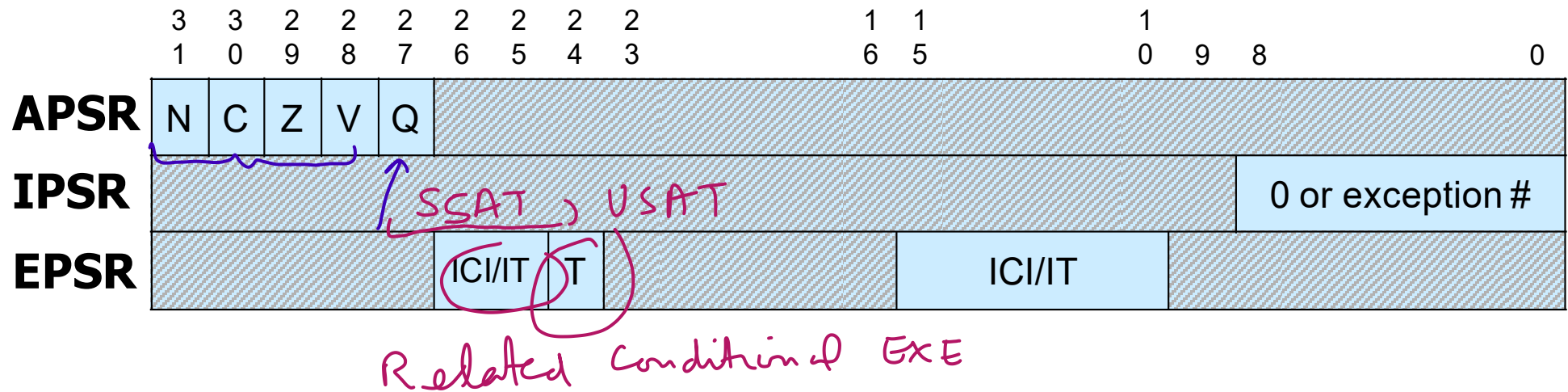
ARM Cortex-M3 Bus

$R1 \leftarrow A$

$R2 \leftarrow \frac{5A}{4}$
 $(A + \frac{A}{4})$



Status Registers (xPSR)



Bits	Name	Description
31	N	Negative (bit 31 of result is 1)
30	C	Unsigned Carry
29	Z	Zero or Equal
28	V	Signed Overflow

Most important for application programming

PSR: Program Status Register

Divided into three bit fields

- Application Program Status Register (APSR)
- Interrupt Program Status Register (IPSR)
- Execution Program Status Register (EPSR)

IPSR holds the exception number for exception processing.

EPSR has the following useful bits.

- ICI/IT bits hold the state information for IT block instructions or instructions that are suspended during interrupt processing.
- Q-bit is the sticky saturation bit and supports two rarely used instructions (SSAT and USAT)

SSAT{cond} Rd, #sat, Rm{, shift}

ADD COND
NEVER
ALWAYS

(NOP)

OPERAND

DSP Application

AD10

32-bit
16-bit
8-bit

0x(1) 0000 0000

0101
5

DSP
INPUT
FILTER

SSAT: Saturate Instruction

- Consider two numbers $0xFFFF\ FFFE$ and $0x0000\ 0002$. A 32-bit mathematical addition would result in $0x1\ 0000\ 0001$ which contain 9 hex digits or 33 binary bits. If the same arithmetic is done in a 32-bit processor, ideally the carry flag will be set and the result in the register will be $0x0000\ 0001$.
- If the operation was done by any comparison instruction this would not cause any harm but during any addition operation this may lead to un-predictable results if the code is not designed to handle such operations. Saturate arithmetic says that when the result crosses the extreme limit the value should be maintained at the respective maximum/minimum (in our case result will be maintained at $0xFFFF\ FFFF$ which is the largest 32-bit number).
- Saturate instructions are very useful in implementing certain DSP algorithms like audio processing.
- Also a new flag field called 'Q' has been added to the ARM processor to show us if there had been any such saturation taken place or the natural result itself was the maximum.

ARM Operating Modes & Register Usage

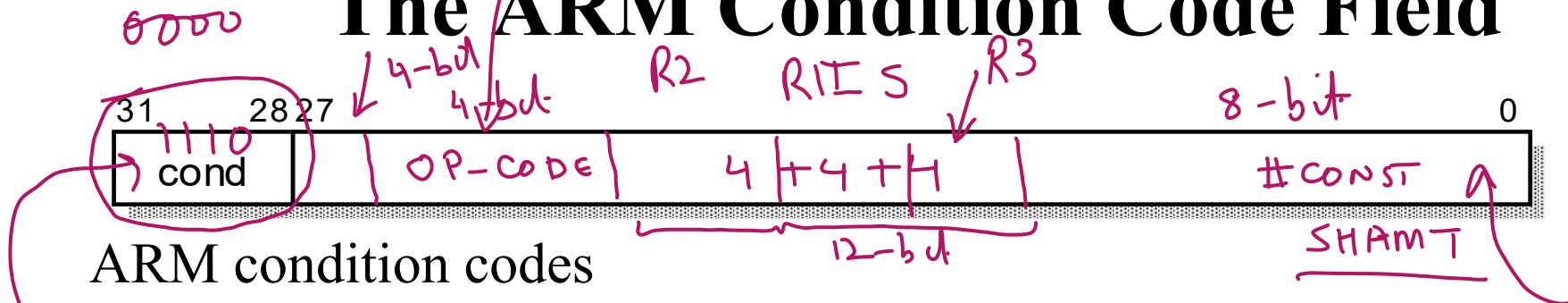
CPSR[4:0]	Mode	Use	Registers
10000	User	Normal user code	user <i>Application</i>
10001	FIQ	Processing fast interrupts	_fiq
10010	IRQ	Processing standard interrupts	_irq
10011	SVC	Processing software interrupts (SWIs)	_svc
10111	Abort	Processing memory faults	_abt
11011	Undef	Handling undefined instruction traps	_und
11111	<u>System</u>	Running privileged operating system tasks	user

Exception vector addresses

Exception	Mode	Vector address
Reset	SVC	0x00000000
Undefined instruction	UND	0x00000004
Software interrupt (SWI)	SVC	0x00000008
Prefetch abort (instruction fetch memory fault)	Abort	0x0000000C
Data abort (data access memory fault)	Abort	0x00000010
IRQ (normal interrupt)	IRQ	0x00000018
FIQ (fast interrupt)	FIQ	0x0000001C

↙ NVIC

The ARM Condition Code Field



ARM condition codes

Opcode [31:28]	Mnemonic extension	Interpretation	Status flag state for execution
0000	EQ	Equal / equals zero	Z set
0001	NE	Not equal	Z clear
0010	CS/HS	Carry set / unsigned higher or same	C set
0011	CC/LO	Carry clear / unsigned lower	C clear
0100	MI	Minus / negative	N set
0101	PL	Plus / positive or zero	N clear
0110	VS	Overflow	V set
0111	VC	No overflow	V clear
1000	HI	Unsigned higher	C set and Z clear
1001	LS	Unsigned lower or same	C clear or Z set
1010	GE	Signed greater than or equal	N equals V
1011	LT	Signed less than	N is not equal to V
1100	GT	Signed greater than	Z clear and N equals V
1101	LE	Signed less than or equal	Z set or N is not equal to V
1110	AL	<u>Always</u>	any
1111	NV	<u>Never (do not use!)</u>	none

16 condition codes

MUL 6 Inst.
cycle

ARM - Interrupt Processing

SENSORS

INT

E-S
M3

12 cycle

PSR
PC

Hardware interrupt request occurs: CPU finishes, suspends or abandons the current instruction and then initiates an exception response sequence.

Exception Response Sequence: CPU stacks the processor state and return address, enables Handler Mode, identifies the requesting device, and transfers control to the corresponding Interrupt Service Routine.

Exception Handler / ISR:

1. Preserve R4-R11 as needed.
2. Transfer data between queue and I/O device.
3. Restore R4-R11 as needed.
4. Return to interrupted code.

Interrupt Complete: Interrupted code continues where it left off as if nothing happened.

Exception Return: Unstack and restore the processor state and mode.

Exception/Interrupt Handler

Exception: a condition that needs to halt the normal sequential flow of instruction execution.

Exception Categories: Reset, SVC Supervisor Call, Fault and Interrupts

Each exception has:

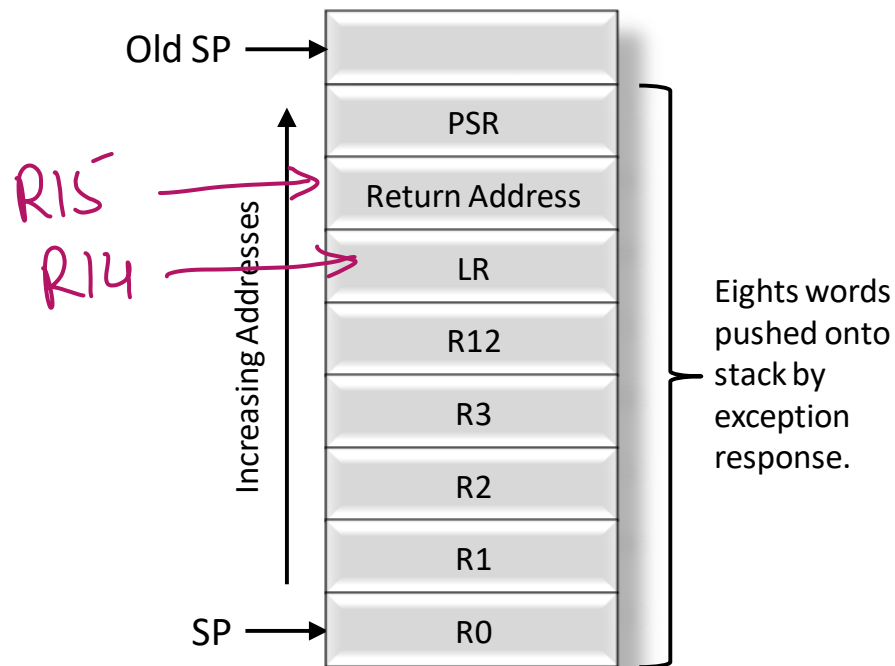
- An exception number
- A priority level
- An exception handler routine
- An entry in the vector table

Exception Response

- Processor state (8 words) stored on stack: CPSR, Return Address, LR, R12, R3 - R0. *Application* PC
- Processor switched (from Thread Mode) to Handler Mode
- $PC \leftarrow \text{vector table}[\text{exception \#}]$

Exception Handlers and Return

An exception handler (ISR) is a software routine that is executed when a specific exception condition occurs.

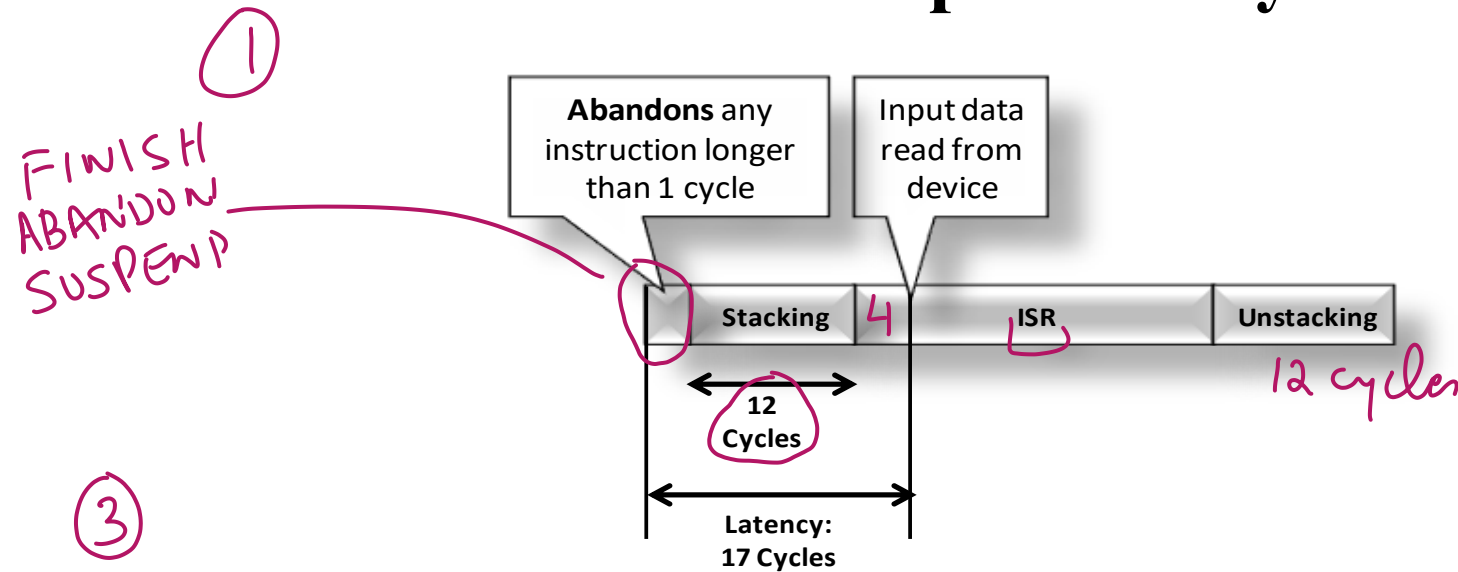


Exception return occurs when in Handler Mode and one of the following instructions is executed:

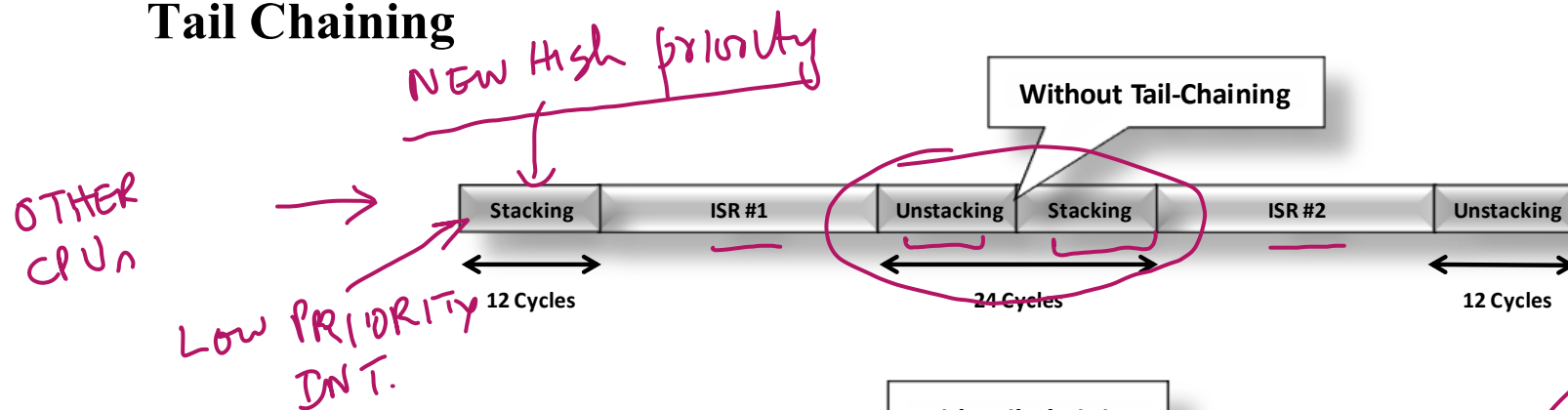
- POP/LDM includes the PC, or
- LDR with PC as destination or
- BX with any register as the source

Interrupt Stacking

Interrupt Latency



Tail Chaining



NESTED INTERRUPT

② LATE ARRIVAL PROCESSING

Interrupt Latency Reduction

Time from interrupt request to the corresponding interrupt handler begins to execute.

1. Suspend or Abandon Instruction Execution:

No need to suspend single cycle instruction but multiple cycle ones.

→ **2. Late Arrival Processing:**

CPU has begun an interrupt response sequence and another high priority interrupt arrive during the stacking operation.

3. Tail Chaining:

In most CPUs when two ISRs execute back to back, the state information is popped off the stack at the end of 1st interrupt only to be pushed back at the beginning of the 2nd interrupt.

M3 completely eliminates this useless pop-push sequence with a technique called tail-chaining, lowering the ISR transition time from 24 down to 6 clock cycles.

CPSIE i ; Enable External Interrupts

CPSID i ; Disable External Interrupts

M3 (Interrupt/Exception) Vector Table

Exception Type	Position	Priority	Comment
	0	-	Initial SP value (loaded on reset)
<u>Reset</u>	1	-3	Power up and warm reset
NMI	2	-2	Non-Maskable Interrupt
<u>Hard Fault</u>	3	-1	(segmentation Fault) when USER APP try to access SYSTEM MEM
Memory Mgmt	4	S e t t a b l e	
Bus Fault	5		Address/Memory-related faults
<u>Usage Fault</u>	6		Undefined instruction
	7-10		Reserved
SVCall	11		Software Interrupt (SVC instruction)
Debug Monitor	12		
	13		Reserved
PendSV	14		
SysTick	15		System Timer Tick
Interrupts	≥16		External; fed through NVIC

Nested Vectored Interrupt Controller

Mapped to addresses E000E100-E000ECFF₁₆

It provides ability to:

- Individually Enable/Disable interrupts from specific devices.
- Establishes relative priorities among the various interrupts.

NVIC INTERRUPTS

IRD

0-4	GPIO Ports A-E	18	Watchdog Timer
5,6	UART 0 & 1	19-24	Timer 0a-2b
7	SSI	25	Analog Comparator
8	I ² C	26-27	Reserved
9	PWM Fault	28	System Control
10-12	PWM Generator 0-2	29	Flash Control
13	Reserved	30-31	Reserved
14-17	ADC Sequence 0-3		

Bit in the interrupt registers